Computer Science
Investigatory Project

HoopsData: Official NBA Finals Portal

Done By: Joshua John Anto

Index

S No.	Content	Page Number
1.	Acknowledgement	2
2.	Aim	3
3.	Brief Explanation	4
4.	Source Code	5 - 37
5.	Output	38 - 44
6.	Conclusion	45
7.	Bibliography	46

Acknowledgment

Through this acknowledgement, I would like to express my sincere gratitude to all those people who were associated with this project and guided us throughout.

We would like to express my sincere gratitude to my teacher **Mr. Munawor Kuppatil** who gave us the opportunity to learn the subject with a practical approach and without whose constant guidance, this project would not have been a success.

We would also like to take this opportunity to thank our parents for their continued support which helped us a lot in the completion of this project.

Aim

To develop a user-friendly, menu-driven Python application ('HoopsData | Official NBA Finals Portal') integrated with a MySQL database (`nbafinalsdb`) for managing, analyzing, and simulating historical NBA Finals records from 1991 to 2026. The system enables efficient handling of franchise offensive and defensive ratings, championship series archives, Finals MVP statistics, NBA arena directories, head-to-head matchup predictions, and best-of-7 play-by-play series simulations, ensuring accurate database CRUD operations and streamlined user interaction.

SOFTWARE SPECIFICATIONS:

- **Programming Language:** Python 3.13 (Latest Version)
- **Code Editor / IDE:** Visual Studio Code (VS Code)
- **Database Management System:** MySQL 8.0 / 9.0 (MySQL Command Line Client)
- **Python Libraries / Modules:** mysql-connector-python, random, datetime

HARDWARE SPECIFICATIONS:

- **EDITION:**
- **PROCESSOR:**
- **Installed RAM:**
- **SYSTEM TYPE:**

Brief Explanation

This project is a Python-based, menu-driven console application designed to manage, query, and simulate NBA Finals data (1991–2026) using a relational MySQL database (`nbafinalsdb`). Upon startup, the portal automatically initializes five normalized tables (`teamstats`, `serieshistory`, `mvpstats`, `arenainfo`, and `simlogs`) and seeds them with historical records. It features an interactive command-line interface with custom ASCII table rendering, allowing users to:

- **Manage Finals Teams & Ratings (teamstats):** Browse all 72 historical Finals teams, filter by season range or conference (East/West), search by franchise or head coach, rank the Top 15 offensive and defensive teams, compare two teams side by side, and add, edit, or delete team records.
- **Explore the Finals Series Archive (serieshistory):** View championship matchups from 1991 to 2026, filter classic 7-game showdowns and 4-game sweeps, generate franchise championship and Finals appearance leaderboards with win percentages, and perform full CRUD operations on series records.
- **Analyze Finals MVP Performances (mvpstats):** Browse and search Finals MVP winners, filter by minimum scoring average (PPG), identify multi-time Finals MVP winners using SQL GROUP BY and HAVING clauses, compare two MVP performances head to head using a weighted Overall Player Score formula, and manage MVP profiles.
- **Manage NBA Arenas & Venues (arenainfo):** Browse NBA stadiums ordered by seating capacity, search venues by city or franchise, calculate active stadium age for classic arenas, and register, update, or remove arena entries.
- **Run Matchup Predictions & 7-Game Simulations (simlogs):** Predict head-to-head series outcomes deterministically using offensive/defensive ratings and pace, or run a randomized best-of-7 game-by-game simulator, automatically logging timestamps, winners, series lengths, and average margins to the database.
- **Generate League Insights & Visual Charts:** Execute multi-table SQL INNER JOIN queries linking teams, coaches, and MVPs, compile decade-by-decade league trend summaries (1990s–2020s), rank the most successful Finals head coaches, render horizontal # bar charts in the terminal, inspect portal row counts, and restore default database records.

The goal is to streamline sports database operations and statistical modeling with a clean, interactive menu interface and reliable MySQL backend integration.

Source Code

```
import mysql.connector
import random
from datetime import datetime

dbhost = "localhost"
dbport = 3306
dbuser = "root"
dbpassword = "Ajoshjohn2021"
dbname = "nbafinalsdb"

def getserverconnection():
    return mysql.connector.connect(
        host=dbhost,
        port=dbport,
        user=dbuser,
        password=dbpassword
    )

def getdbconnection():
    return mysql.connector.connect(
        host=dbhost,
        port=dbport,
        user=dbuser,
        password=dbpassword,
        database=dbname
    )

def initdatabase():
    try:
        conn = getserverconnection()
        cursor = conn.cursor()
        cursor.execute("CREATE DATABASE IF NOT EXISTS " + dbname)
        cursor.close()
        conn.close()

        conn = getdbconnection()
        cursor = conn.cursor()

        cursor.execute(
            "CREATE TABLE IF NOT EXISTS teamstats ("
            "teamid INT PRIMARY KEY, "
            "teamname VARCHAR(100) NOT NULL UNIQUE, "
            "seasonyear INT NOT NULL, "
            "conference VARCHAR(20) NOT NULL, "
            "offrating DECIMAL(5,2) NOT NULL, "
            "defrating DECIMAL(5,2) NOT NULL, "
            "pace DECIMAL(5,2) NOT NULL, "
            "coach VARCHAR(100) NOT NULL)"
        )

        cursor.execute(
            "CREATE TABLE IF NOT EXISTS serieshistory ("
            "year INT PRIMARY KEY, "
```

```
            "champion VARCHAR(100) NOT NULL, "
            "runnerup VARCHAR(100) NOT NULL, "
            "seriesresult VARCHAR(20) NOT NULL, "
            "finalsmvp VARCHAR(100) NOT NULL, "
            "totalgames INT NOT NULL, "
            "avgattendance INT NOT NULL)"
        )

        cursor.execute(
            "CREATE TABLE IF NOT EXISTS mvpstats ("
            "year INT PRIMARY KEY, "
            "playername VARCHAR(100) NOT NULL, "
            "team VARCHAR(100) NOT NULL, "
            "ppg DECIMAL(4,1) NOT NULL, "
            "rpg DECIMAL(4,1) NOT NULL, "
            "apg DECIMAL(4,1) NOT NULL, "
            "fgpct DECIMAL(4,1) NOT NULL)"
        )

        cursor.execute(
            "CREATE TABLE IF NOT EXISTS arenainfo ("
            "arenaid INT PRIMARY KEY, "
            "arenaname VARCHAR(100) NOT NULL, "
            "city VARCHAR(100) NOT NULL, "
            "franchise VARCHAR(100) NOT NULL, "
            "capacity INT NOT NULL, "
            "openedyear INT NOT NULL)"
        )

        cursor.execute(
            "CREATE TABLE IF NOT EXISTS simlogs ("
            "simid INT PRIMARY KEY, "
            "simdate VARCHAR(50) NOT NULL, "
            "teama VARCHAR(100) NOT NULL, "
            "teamb VARCHAR(100) NOT NULL, "
            "winner VARCHAR(100) NOT NULL, "
            "predictedgames INT NOT NULL, "
            "margin DECIMAL(5,2) NOT NULL)"
        )

        cursor.execute("SELECT COUNT(*) FROM teamstats")
        if cursor.fetchone()[0] == 0:
            teamdata = [
                (1, "1991 Chicago Bulls", 1991, "East", 114.6, 105.2, 95.6, "Phil Jackson"),
                (2, "1991 Los Angeles Lakers", 1991, "West", 112.1, 106.4, 94.8, "Mike Dunleavy"),
                (3, "1992 Chicago Bulls", 1992, "East", 115.5, 104.5, 94.4, "Phil Jackson"),
                (4, "1992 Portland Trail Blazers", 1992, "West", 111.4, 104.2, 99.1, "Rick
Adelman"),
                (5, "1993 Chicago Bulls", 1993, "East", 112.9, 106.1, 92.5, "Phil Jackson"),
                (6, "1993 Phoenix Suns", 1993, "West", 113.3, 106.7, 99.3, "Paul Westphal"),
                (7, "1994 Houston Rockets", 1994, "West", 105.9, 101.4, 94.8, "Rudy Tomjanovich"),
                (8, "1994 New York Knicks", 1994, "East", 105.7, 98.2, 91.5, "Pat Riley"),
                (9, "1995 Houston Rockets", 1995, "West", 109.7, 107.4, 94.2, "Rudy Tomjanovich"),
                (10, "1995 Orlando Magic", 1995, "East", 115.1, 107.8, 95.8, "Brian Hill"),
                (11, "1996 Chicago Bulls", 1996, "East", 115.2, 101.8, 91.1, "Phil Jackson"),
                (12, "1996 Seattle SuperSonics", 1996, "West", 110.3, 102.1, 93.8, "George Karl"),
                (13, "1997 Chicago Bulls", 1997, "East", 114.4, 102.4, 90.0, "Phil Jackson"),
```

```
            (14, "1997 Utah Jazz", 1997, "West", 113.6, 104.0, 90.4, "Jerry Sloan"),
            (15, "1998 Chicago Bulls", 1998, "East", 107.7, 99.8, 89.0, "Phil Jackson"),
            (16, "1998 Utah Jazz", 1998, "West", 112.7, 105.4, 89.2, "Jerry Sloan"),
            (17, "1999 San Antonio Spurs", 1999, "West", 104.0, 95.0, 88.6, "Gregg Popovich"),
            (18, "1999 New York Knicks", 1999, "East", 101.2, 98.4, 86.8, "Jeff Van Gundy"),
            (19, "2000 Los Angeles Lakers", 2000, "West", 107.3, 98.2, 93.3, "Phil Jackson"),
            (20, "2000 Indiana Pacers", 2000, "East", 108.5, 103.6, 92.4, "Larry Bird"),
            (21, "2001 Los Angeles Lakers", 2001, "West", 108.4, 104.8, 91.7, "Phil Jackson"),
            (22, "2001 Philadelphia 76ers", 2001, "East", 101.8, 98.9, 90.6, "Larry Brown"),
            (23, "2002 Los Angeles Lakers", 2002, "West", 109.4, 101.7, 92.1, "Phil Jackson"),
            (24, "2002 New Jersey Nets", 2002, "East", 101.1, 99.5, 91.8, "Byron Scott"),
            (25, "2003 San Antonio Spurs", 2003, "West", 105.6, 99.7, 90.0, "Gregg Popovich"),
            (26, "2003 New Jersey Nets", 2003, "East", 100.0, 96.0, 91.2, "Byron Scott"),
            (27, "2004 Detroit Pistons", 2004, "East", 102.0, 95.4, 87.9, "Larry Brown"),
            (28, "2004 Los Angeles Lakers", 2004, "West", 105.5, 103.2, 91.8, "Phil Jackson"),
            (29, "2005 San Antonio Spurs", 2005, "West", 107.5, 98.8, 88.9, "Gregg Popovich"),
            (30, "2005 Detroit Pistons", 2005, "East", 103.9, 98.1, 86.8, "Larry Brown"),
            (31, "2006 Miami Heat", 2006, "East", 108.7, 104.3, 91.6, "Pat Riley"),
            (32, "2006 Dallas Mavericks", 2006, "West", 111.8, 105.0, 89.6, "Avery Johnson"),
            (33, "2007 San Antonio Spurs", 2007, "West", 109.2, 99.9, 89.8, "Gregg Popovich"),
            (34, "2007 Cleveland Cavaliers", 2007, "East", 103.8, 101.3, 90.8, "Mike Brown"),
            (35, "2008 Boston Celtics", 2008, "East", 110.2, 98.9, 90.9, "Doc Rivers"),
            (36, "2008 Los Angeles Lakers", 2008, "West", 113.0, 105.5, 95.6, "Phil Jackson"),
            (37, "2009 Los Angeles Lakers", 2009, "West", 112.8, 104.7, 94.3, "Phil Jackson"),
            (38, "2009 Orlando Magic", 2009, "East", 108.6, 101.9, 92.2, "Stan Van Gundy"),
            (39, "2010 Los Angeles Lakers", 2010, "West", 108.8, 103.7, 92.8, "Phil Jackson"),
            (40, "2010 Boston Celtics", 2010, "East", 107.7, 103.8, 91.6, "Doc Rivers"),
            (41, "2011 Dallas Mavericks", 2011, "West", 109.7, 105.0, 91.3, "Rick Carlisle"),
            (42, "2011 Miami Heat", 2011, "East", 111.7, 103.5, 90.9, "Erik Spoelstra"),
            (43, "2012 Miami Heat", 2012, "East", 108.5, 100.2, 91.2, "Erik Spoelstra"),
            (44, "2012 Oklahoma City Thunder", 2012, "West", 109.8, 103.2, 93.0, "Scott
Brooks"),
            (45, "2013 Miami Heat", 2013, "East", 112.3, 103.1, 90.7, "Erik Spoelstra"),
            (46, "2013 San Antonio Spurs", 2013, "West", 108.1, 101.6, 94.2, "Gregg
Popovich"),
            (47, "2014 San Antonio Spurs", 2014, "West", 110.5, 102.4, 95.0, "Gregg
Popovich"),
            (48, "2014 Miami Heat", 2014, "East", 109.0, 105.8, 91.2, "Erik Spoelstra"),
            (49, "2015 Golden State Warriors", 2015, "West", 111.6, 101.4, 98.3, "Steve
Kerr"),
            (50, "2015 Cleveland Cavaliers", 2015, "East", 109.8, 106.3, 92.3, "David Blatt"),
            (51, "2016 Cleveland Cavaliers", 2016, "East", 110.9, 104.2, 93.3, "Tyronn Lue"),
            (52, "2016 Golden State Warriors", 2016, "West", 114.5, 103.8, 99.3, "Steve
Kerr"),
            (53, "2017 Golden State Warriors", 2017, "West", 115.6, 102.5, 99.8, "Steve
Kerr"),
            (54, "2017 Cleveland Cavaliers", 2017, "East", 113.6, 108.0, 96.2, "Tyronn Lue"),
            (55, "2018 Golden State Warriors", 2018, "West", 113.6, 107.7, 99.6, "Steve
Kerr"),
            (56, "2018 Cleveland Cavaliers", 2018, "East", 111.1, 110.6, 98.0, "Tyronn Lue"),
            (57, "2019 Toronto Raptors", 2019, "East", 113.1, 107.1, 100.2, "Nick Nurse"),
            (58, "2019 Golden State Warriors", 2019, "West", 115.9, 109.0, 100.9, "Steve
Kerr"),
            (59, "2020 Los Angeles Lakers", 2020, "West", 112.0, 105.6, 100.9, "Frank Vogel"),
            (60, "2020 Miami Heat", 2020, "East", 112.5, 109.8, 98.3, "Erik Spoelstra"),
            (61, "2021 Milwaukee Bucks", 2021, "East", 117.2, 111.0, 102.2, "Mike
Budenholzer"),
            (62, "2021 Phoenix Suns", 2021, "West", 117.2, 111.3, 97.2, "Monty Williams"),
```

```
                (63, "2022 Golden State Warriors", 2022, "West", 112.5, 106.6, 98.4, "Steve
Kerr"),
                (64, "2022 Boston Celtics", 2022, "East", 114.4, 106.9, 96.6, "Ime Udoka"),
                (65, "2023 Denver Nuggets", 2023, "West", 117.6, 113.5, 98.1, "Michael Malone"),
                (66, "2023 Miami Heat", 2023, "East", 113.0, 112.8, 96.3, "Erik Spoelstra"),
                (67, "2024 Boston Celtics", 2024, "East", 122.2, 109.2, 97.9, "Joe Mazzulla"),
                (68, "2024 Dallas Mavericks", 2024, "West", 117.0, 114.9, 100.1, "Jason Kidd"),
                (69, "2025 Oklahoma City Thunder", 2025, "West", 118.5, 108.2, 100.4, "Mark
Daigneault"),
                (70, "2025 Indiana Pacers", 2025, "East", 119.8, 115.1, 101.8, "Rick Carlisle"),
                (71, "2026 New York Knicks", 2026, "East", 119.8, 111.2, 97.8, "Mike Brown"),
                (72, "2026 San Antonio Spurs", 2026, "West", 118.4, 110.1, 99.6, "Mitch Johnson")
            ]
            cursor.executemany(
                "INSERT INTO teamstats VALUES (%s, %s, %s, %s, %s, %s, %s, %s)",
                teamdata
            )
        else:
            cursor.execute(
                "REPLACE INTO teamstats VALUES (%s, %s, %s, %s, %s, %s, %s, %s)",
                (71, "2026 New York Knicks", 2026, "East", 119.8, 111.2, 97.8, "Mike Brown")
            )
            cursor.execute(
                "REPLACE INTO teamstats VALUES (%s, %s, %s, %s, %s, %s, %s, %s)",
                (72, "2026 San Antonio Spurs", 2026, "West", 118.4, 110.1, 99.6, "Mitch Johnson")
            )

        cursor.execute("SELECT COUNT(*) FROM serieshistory")
        if cursor.fetchone()[0] == 0:
            historydata = [
                (1991, "Chicago Bulls", "Los Angeles Lakers", "4 to 1", "Michael Jordan", 5,
17950),
                (1992, "Chicago Bulls", "Portland Trail Blazers", "4 to 2", "Michael Jordan", 6,
18100),
                (1993, "Chicago Bulls", "Phoenix Suns", "4 to 2", "Michael Jordan", 6, 18800),
                (1994, "Houston Rockets", "New York Knicks", "4 to 3", "Hakeem Olajuwon", 7,
17800),
                (1995, "Houston Rockets", "Orlando Magic", "4 to 0", "Hakeem Olajuwon", 4, 17100),
                (1996, "Chicago Bulls", "Seattle SuperSonics", "4 to 2", "Michael Jordan", 6,
20200),
                (1997, "Chicago Bulls", "Utah Jazz", "4 to 2", "Michael Jordan", 6, 21800),
                (1998, "Chicago Bulls", "Utah Jazz", "4 to 2", "Michael Jordan", 6, 22100),
                (1999, "San Antonio Spurs", "New York Knicks", "4 to 1", "Tim Duncan", 5, 25400),
                (2000, "Los Angeles Lakers", "Indiana Pacers", "4 to 2", "Shaquille O'Neal", 6,
18900),
                (2001, "Los Angeles Lakers", "Philadelphia 76ers", "4 to 1", "Shaquille O'Neal",
5, 19800),
                (2002, "Los Angeles Lakers", "New Jersey Nets", "4 to 0", "Shaquille O'Neal", 4,
19400),
                (2003, "San Antonio Spurs", "New Jersey Nets", "4 to 2", "Tim Duncan", 6, 18700),
                (2004, "Detroit Pistons", "Los Angeles Lakers", "4 to 1", "Chauncey Billups", 5,
20900),
                (2005, "San Antonio Spurs", "Detroit Pistons", "4 to 3", "Tim Duncan", 7, 20400),
                (2006, "Miami Heat", "Dallas Mavericks", "4 to 2", "Dwyane Wade", 6, 20100),
                (2007, "San Antonio Spurs", "Cleveland Cavaliers", "4 to 0", "Tony Parker", 4,
19700),
                (2008, "Boston Celtics", "Los Angeles Lakers", "4 to 2", "Paul Pierce", 6, 18800),
                (2009, "Los Angeles Lakers", "Orlando Magic", "4 to 1", "Kobe Bryant", 5, 18400),
```

```
                (2010, "Los Angeles Lakers", "Boston Celtics", "4 to 3", "Kobe Bryant", 7, 18900),
                (2011, "Dallas Mavericks", "Miami Heat", "4 to 2", "Dirk Nowitzki", 6, 20050),
                (2012, "Miami Heat", "Oklahoma City Thunder", "4 to 1", "LeBron James", 5, 19300),
                (2013, "Miami Heat", "San Antonio Spurs", "4 to 3", "LeBron James", 7, 19500),
                (2014, "San Antonio Spurs", "Miami Heat", "4 to 1", "Kawhi Leonard", 5, 19100),
                (2015, "Golden State Warriors", "Cleveland Cavaliers", "4 to 2", "Andre Iguodala",
6, 20100),
                (2016, "Cleveland Cavaliers", "Golden State Warriors", "4 to 3", "LeBron James",
7, 20200),
                (2017, "Golden State Warriors", "Cleveland Cavaliers", "4 to 1", "Kevin Durant",
5, 20150),
                (2018, "Golden State Warriors", "Cleveland Cavaliers", "4 to 0", "Kevin Durant",
4, 20050),
                (2019, "Toronto Raptors", "Golden State Warriors", "4 to 2", "Kawhi Leonard", 6,
19800),
                (2020, "Los Angeles Lakers", "Miami Heat", "4 to 2", "LeBron James", 6, 0),
                (2021, "Milwaukee Bucks", "Phoenix Suns", "4 to 2", "Giannis Antetokounmpo", 6,
17100),
                (2022, "Golden State Warriors", "Boston Celtics", "4 to 2", "Stephen Curry", 6,
18600),
                (2023, "Denver Nuggets", "Miami Heat", "4 to 1", "Nikola Jokic", 5, 19600),
                (2024, "Boston Celtics", "Dallas Mavericks", "4 to 1", "Jaylen Brown", 5, 19700),
                (2025, "Oklahoma City Thunder", "Indiana Pacers", "4 to 3", "Shai Gilgeous
Alexander", 7, 18200),
                (2026, "New York Knicks", "San Antonio Spurs", "4 to 1", "Jalen Brunson", 5,
19150)
            ]
            cursor.executemany(
                "INSERT INTO serieshistory VALUES (%s, %s, %s, %s, %s, %s, %s)",
                historydata
            )
        else:
            cursor.execute(
                "REPLACE INTO serieshistory VALUES (%s, %s, %s, %s, %s, %s, %s)",
                (2026, "New York Knicks", "San Antonio Spurs", "4 to 1", "Jalen Brunson", 5,
19150)
            )

        cursor.execute("SELECT COUNT(*) FROM mvpstats")
        if cursor.fetchone()[0] == 0:
            mvpdata = [
                (1991, "Michael Jordan", "Chicago Bulls", 31.2, 6.6, 11.4, 55.8),
                (1992, "Michael Jordan", "Chicago Bulls", 35.8, 4.8, 6.5, 52.6),
                (1993, "Michael Jordan", "Chicago Bulls", 41.0, 8.5, 6.3, 50.8),
                (1994, "Hakeem Olajuwon", "Houston Rockets", 26.9, 9.1, 3.6, 50.0),
                (1995, "Hakeem Olajuwon", "Houston Rockets", 32.8, 11.5, 5.5, 48.3),
                (1996, "Michael Jordan", "Chicago Bulls", 27.3, 5.3, 4.2, 41.5),
                (1997, "Michael Jordan", "Chicago Bulls", 32.3, 7.0, 6.0, 45.6),
                (1998, "Michael Jordan", "Chicago Bulls", 33.5, 4.0, 2.3, 42.7),
                (1999, "Tim Duncan", "San Antonio Spurs", 27.4, 14.0, 2.4, 53.7),
                (2000, "Shaquille O'Neal", "Los Angeles Lakers", 38.0, 16.7, 2.3, 61.1),
                (2001, "Shaquille O'Neal", "Los Angeles Lakers", 33.0, 15.8, 4.8, 57.3),
                (2002, "Shaquille O'Neal", "Los Angeles Lakers", 36.3, 12.3, 3.8, 59.5),
                (2003, "Tim Duncan", "San Antonio Spurs", 24.2, 17.0, 5.3, 49.5),
                (2004, "Chauncey Billups", "Detroit Pistons", 21.0, 3.2, 5.2, 50.9),
                (2005, "Tim Duncan", "San Antonio Spurs", 20.6, 14.1, 2.1, 41.9),
                (2006, "Dwyane Wade", "Miami Heat", 34.7, 7.8, 3.8, 46.8),
                (2007, "Tony Parker", "San Antonio Spurs", 24.5, 5.0, 3.3, 56.8),
```

```
        (2008, "Paul Pierce", "Boston Celtics", 21.8, 4.5, 6.3, 43.2),
        (2009, "Kobe Bryant", "Los Angeles Lakers", 32.4, 5.6, 7.4, 43.0),
        (2010, "Kobe Bryant", "Los Angeles Lakers", 28.6, 8.0, 3.9, 40.5),
        (2011, "Dirk Nowitzki", "Dallas Mavericks", 26.0, 9.7, 2.0, 41.6),
        (2012, "LeBron James", "Miami Heat", 28.6, 10.2, 7.4, 47.2),
        (2013, "LeBron James", "Miami Heat", 25.3, 10.9, 7.0, 44.7),
        (2014, "Kawhi Leonard", "San Antonio Spurs", 17.8, 6.4, 2.0, 61.2),
        (2015, "Andre Iguodala", "Golden State Warriors", 16.3, 5.8, 4.0, 52.1),
        (2016, "LeBron James", "Cleveland Cavaliers", 29.7, 11.3, 8.9, 49.4),
        (2017, "Kevin Durant", "Golden State Warriors", 35.2, 8.2, 5.4, 55.6),
        (2018, "Kevin Durant", "Golden State Warriors", 28.8, 10.8, 7.5, 52.6),
        (2019, "Kawhi Leonard", "Toronto Raptors", 28.5, 9.8, 4.2, 43.4),
        (2020, "LeBron James", "Los Angeles Lakers", 29.8, 11.8, 8.5, 59.1),
        (2021, "Giannis Antetokounmpo", "Milwaukee Bucks", 35.2, 13.2, 5.0, 61.8),
        (2022, "Stephen Curry", "Golden State Warriors", 31.2, 6.0, 5.0, 48.2),
        (2023, "Nikola Jokic", "Denver Nuggets", 30.2, 14.0, 7.2, 58.3),
        (2024, "Jaylen Brown", "Boston Celtics", 20.8, 5.4, 5.0, 44.0),
        (2025, "Shai Gilgeous Alexander", "Oklahoma City Thunder", 30.3, 4.6, 5.6, 44.3),
        (2026, "Jalen Brunson", "New York Knicks", 32.6, 4.2, 4.6, 42.1)
    ]
    cursor.executemany(
        "INSERT INTO mvpstats VALUES (%s, %s, %s, %s, %s, %s, %s)",
        mvpdata
    )
else:
    cursor.execute(
        "REPLACE INTO mvpstats VALUES (%s, %s, %s, %s, %s, %s, %s)",
        (2025, "Shai Gilgeous Alexander", "Oklahoma City Thunder", 30.3, 4.6, 5.6, 44.3)
    )
    cursor.execute(
        "REPLACE INTO mvpstats VALUES (%s, %s, %s, %s, %s, %s, %s)",
        (2026, "Jalen Brunson", "New York Knicks", 32.6, 4.2, 4.6, 42.1)
    )

cursor.execute("SELECT COUNT(*) FROM arenainfo")
if cursor.fetchone()[0] == 0:
    arenadata = [
        (1, "United Center", "Chicago", "Chicago Bulls", 20917, 1994),
        (2, "Crypto.com Arena", "Los Angeles", "Los Angeles Lakers", 18997, 1999),
        (3, "TD Garden", "Boston", "Boston Celtics", 19156, 1995),
        (4, "Frost Bank Center", "San Antonio", "San Antonio Spurs", 18418, 2002),
        (5, "Kaseya Center", "Miami", "Miami Heat", 19600, 1999),
        (6, "Chase Center", "San Francisco", "Golden State Warriors", 18064, 2019),
        (7, "Rocket Arena", "Cleveland", "Cleveland Cavaliers", 19432, 1994),
        (8, "American Airlines Center", "Dallas", "Dallas Mavericks", 19200, 2001),
        (9, "Little Caesars Arena", "Detroit", "Detroit Pistons", 20332, 2017),
        (10, "Toyota Center", "Houston", "Houston Rockets", 18055, 2003),
        (11, "Scotiabank Arena", "Toronto", "Toronto Raptors", 19800, 1999),
        (12, "Fiserv Forum", "Milwaukee", "Milwaukee Bucks", 17385, 2018),
        (13, "Ball Arena", "Denver", "Denver Nuggets", 19520, 1999),
        (14, "Paycom Center", "Oklahoma City", "Oklahoma City Thunder", 18203, 2002),
        (15, "Gainbridge Fieldhouse", "Indianapolis", "Indiana Pacers", 17923, 1999),
        (16, "Madison Square Garden", "New York", "New York Knicks", 19812, 1968),
        (17, "Xfinity Mobile Arena", "Philadelphia", "Philadelphia 76ers", 20478, 1996),
        (18, "Footprint Center", "Phoenix", "Phoenix Suns", 17071, 1992),
        (19, "Kia Center", "Orlando", "Orlando Magic", 18846, 2010),
        (20, "Delta Center", "Salt Lake City", "Utah Jazz", 18306, 1991),
        (21, "Moda Center", "Portland", "Portland Trail Blazers", 19393, 1995),
```

```
                (22, "Barclays Center", "Brooklyn", "Brooklyn Nets", 17732, 2012)
            ]
            cursor.executemany(
                "INSERT INTO arenainfo VALUES (%s, %s, %s, %s, %s, %s)",
                arenadata
            )

        conn.commit()
        cursor.close()
        conn.close()
    except Exception as e:
        print("Portal initialization error:", e)

def printline(length):
    print("+" + ("=" * (length - 2)) + "+")

def printheader(title):
    print()
    printline(98)
    print("|" + title.center(96) + "|")
    printline(98)

def pausescreen():
    print()
    input("Press Enter to go back...")

def readint(prompt, minval, maxval):
    while True:
        raw = input(prompt).strip()
        try:
            val = int(raw)
            if val < minval or val > maxval:
                print("Please enter a number between", minval, "and", maxval)
            else:
                return val
        except ValueError:
            print("Invalid input. Please enter a whole number.")

def readfloat(prompt, minval, maxval):
    while True:
        raw = input(prompt).strip()
        try:
            val = float(raw)
            if val < minval or val > maxval:
                print("Please enter a value between", minval, "and", maxval)
            else:
                return round(val, 2)
        except ValueError:
            print("Invalid input. Please enter a decimal number.")

def readtext(prompt):
    while True:
        val = input(prompt).strip()
        if len(val) > 0:
            return val
        print("This field cannot be empty. Please try again.")
```

```
def formatcell(value, width):
    text = str(value).replace("\n", " ").strip()
    if len(text) > width:
        return text[:width]
    return text.ljust(width)

def builddivider(widths):
    parts = ["+"]
    for w in widths:
        parts.append("=" * (w + 2))
        parts.append("+")
    return "".join(parts)

def printtable(headers, widths, rows):
    divider = builddivider(widths)
    totalwidth = len(divider)
    print(divider)
    headerparts = ["|"]
    for i in range(len(headers)):
        headerparts.append(" " + formatcell(headers[i], widths[i]) + " |")
    print("".join(headerparts))
    print(divider)
    if len(rows) == 0:
        emptytext = "No matching results found.".ljust(totalwidth - 4)
        print("| " + emptytext + " |")
    else:
        for row in rows:
            rowparts = ["|"]
            for i in range(len(widths)):
                rowparts.append(" " + formatcell(row[i], widths[i]) + " |")
            print("".join(rowparts))
    print(divider)
    print("Showing", len(rows), "results")

def viewallteams():
    printheader("ALL NBA FINALS TEAMS DIRECTORY")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT teamid, teamname, seasonyear, conference, offrating, defrating, "
        "ROUND(offrating - defrating, 2) AS netrating, pace, coach "
        "FROM teamstats ORDER BY seasonyear ASC, teamid ASC"
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["ID", "Team Name", "Year", "Conf", "Off", "Def", "Net", "Pace", "Head Coach"]
    widths = [2, 27, 4, 4, 5, 5, 5, 5, 16]
    printtable(headers, widths, rows)

def viewteamsbyyear():
    printheader("BROWSE TEAMS BY SEASON YEARS")
    startyear = readint("Enter Starting Season Year (1991 to 2030): ", 1991, 2030)
    endyear = readint("Enter Ending Season Year (1991 to 2030): ", startyear, 2030)
    conn = getdbconnection()
    cursor = conn.cursor()
```

```
    cursor.execute(
        "SELECT teamid, teamname, seasonyear, conference, offrating, defrating, "
        "ROUND(offrating - defrating, 2), coach "
        "FROM teamstats WHERE seasonyear BETWEEN %s AND %s ORDER BY seasonyear DESC",
        (startyear, endyear)
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["ID", "Team Name", "Year", "Conf", "Offense", "Defense", "Overall", "Head Coach"]
    widths = [2, 27, 4, 4, 7, 7, 7, 16]
    printtable(headers, widths, rows)

def viewteamsbyconference():
    printheader("BROWSE TEAMS BY CONFERENCE")
    print("1. Eastern Conference")
    print("2. Western Conference")
    choice = readint("Select a conference (1 or 2): ", 1, 2)
    conf = "East" if choice == 1 else "West"
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT teamid, teamname, seasonyear, offrating, defrating, "
        "ROUND(offrating - defrating, 2), pace, coach "
        "FROM teamstats WHERE conference = %s ORDER BY seasonyear DESC",
        (conf,)
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["ID", "Team Name", "Year", "Offense", "Defense", "Overall", "Pace", "Head Coach"]
    widths = [2, 27, 4, 7, 7, 7, 5, 16]
    printtable(headers, widths, rows)

def searchteambyname():
    printheader("SEARCH TEAMS BY FRANCHISE OR COACH")
    keyword = readtext("Enter a team name or head coach name: ")
    conn = getdbconnection()
    cursor = conn.cursor()
    pattern = "%" + keyword + "%"
    cursor.execute(
        "SELECT teamid, teamname, seasonyear, conference, offrating, defrating, "
        "ROUND(offrating - defrating, 2), coach "
        "FROM teamstats WHERE teamname LIKE %s OR coach LIKE %s ORDER BY seasonyear DESC",
        (pattern, pattern)
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["ID", "Team Name", "Year", "Conf", "Offense", "Defense", "Overall", "Head Coach"]
    widths = [2, 27, 4, 4, 7, 7, 7, 16]
    printtable(headers, widths, rows)

def topoffensiveteams():
    printheader("TOP 15 HIGHEST SCORING FINALS TEAMS")
    conn = getdbconnection()
    cursor = conn.cursor()
```

```
    cursor.execute(
        "SELECT teamid, teamname, seasonyear, conference, offrating, defrating, "
        "ROUND(offrating - defrating, 2), coach "
        "FROM teamstats ORDER BY offrating DESC LIMIT 15"
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["ID", "Team Name", "Year", "Conf", "Offense", "Defense", "Overall", "Head Coach"]
    widths = [2, 27, 4, 4, 7, 7, 7, 16]
    printtable(headers, widths, rows)

def topdefensiveteams():
    printheader("TOP 15 BEST DEFENSIVE FINALS TEAMS")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT teamid, teamname, seasonyear, conference, defrating, offrating, "
        "ROUND(offrating - defrating, 2), coach "
        "FROM teamstats ORDER BY defrating ASC LIMIT 15"
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["ID", "Team Name", "Year", "Conf", "Defense", "Offense", "Overall", "Head Coach"]
    widths = [2, 27, 4, 4, 7, 7, 7, 16]
    printtable(headers, widths, rows)

def comparetwoteams():
    printheader("SIDE BY SIDE TEAM COMPARISON")
    id1 = readint("Enter First Team ID: ", 1, 9999)
    id2 = readint("Enter Second Team ID: ", 1, 9999)
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT teamname, seasonyear, conference, offrating, defrating, pace, coach "
        "FROM teamstats WHERE teamid = %s",
        (id1,)
    )
    t1 = cursor.fetchone()
    cursor.execute(
        "SELECT teamname, seasonyear, conference, offrating, defrating, pace, coach "
        "FROM teamstats WHERE teamid = %s",
        (id2,)
    )
    t2 = cursor.fetchone()
    cursor.close()
    conn.close()

    if not t1 or not t2:
        print("One or both Team IDs could not be found.")
        return

    net1 = round(float(t1[3]) - float(t1[4]), 2)
    net2 = round(float(t2[3]) - float(t2[4]), 2)
    headers = ["Team Attribute", t1[0], t2[0]]
    widths = [20, 27, 27]
```

```
    rows = [
        ("Season Year", t1[1], t2[1]),
        ("Conference", t1[2], t2[2]),
        ("Head Coach", t1[6], t2[6]),
        ("Offensive Rating", t1[3], t2[3]),
        ("Defensive Rating", t1[4], t2[4]),
        ("Overall Net Rating", net1, net2),
        ("Game Pace", t1[5], t2[5])
    ]
    printtable(headers, widths, rows)

def addnewteam():
    printheader("ADD A NEW FINALS TEAM")
    seasonyear = readint("Enter Season Year (1950 to 2035): ", 1950, 2035)
    franchise = readtext("Enter Team Franchise Name (for example, New York Knicks): ")
    teamname = str(seasonyear) + " " + franchise
    print("Select Conference: 1. East  2. West")
    confchoice = readint("Enter choice (1 or 2): ", 1, 2)
    conference = "East" if confchoice == 1 else "West"
    offrating = readfloat("Enter Offensive Rating (80.0 to 140.0): ", 80.0, 140.0)
    defrating = readfloat("Enter Defensive Rating (80.0 to 140.0): ", 80.0, 140.0)
    pace = readfloat("Enter Team Pace (75.0 to 120.0): ", 75.0, 120.0)
    coach = readtext("Enter Head Coach Name: ")

    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute("SELECT COALESCE(MAX(teamid), 0) + 1 FROM teamstats")
    newid = cursor.fetchone()[0]
    try:
        cursor.execute(
            "INSERT INTO teamstats VALUES (%s, %s, %s, %s, %s, %s, %s, %s)",
            (newid, teamname, seasonyear, conference, offrating, defrating, pace, coach)
        )
        conn.commit()
        print("Added team:", teamname, "with ID:", newid)
    except Exception as e:
        print("Could not add team:", e)
    cursor.close()
    conn.close()

def updateteamratings():
    printheader("EDIT TEAM RATINGS AND PACE")
    teamid = readint("Enter Team ID to edit: ", 1, 9999)
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute("SELECT teamname, offrating, defrating, pace FROM teamstats WHERE teamid = %s",
(teamid,))
    row = cursor.fetchone()
    if not row:
        print("No team found with ID:", teamid)
        cursor.close()
        conn.close()
        return
    print("Selected Team:", row[0])
    print("Current Offense:", row[1], "| Current Defense:", row[2], "| Current Pace:", row[3])
    newoff = readfloat("Enter New Offensive Rating (80.0 to 140.0): ", 80.0, 140.0)
    newdef = readfloat("Enter New Defensive Rating (80.0 to 140.0): ", 80.0, 140.0)
```

```
    newpace = readfloat("Enter New Pace (75.0 to 120.0): ", 75.0, 120.0)
    cursor.execute(
        "UPDATE teamstats SET offrating = %s, defrating = %s, pace = %s WHERE teamid = %s",
        (newoff, newdef, newpace, teamid)
    )
    conn.commit()
    print("Updated ratings for", row[0])
    cursor.close()
    conn.close()

def updateteamcoach():
    printheader("CHANGE TEAM HEAD COACH")
    teamid = readint("Enter Team ID to edit: ", 1, 9999)
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute("SELECT teamname, coach FROM teamstats WHERE teamid = %s", (teamid,))
    row = cursor.fetchone()
    if not row:
        print("No team found with ID:", teamid)
        cursor.close()
        conn.close()
        return
    print("Selected Team:", row[0], "| Current Head Coach:", row[1])
    newcoach = readtext("Enter New Head Coach Name: ")
    cursor.execute("UPDATE teamstats SET coach = %s WHERE teamid = %s", (newcoach, teamid))
    conn.commit()
    print("Updated head coach for", row[0])
    cursor.close()
    conn.close()

def deleteteamrecord():
    printheader("REMOVE A TEAM FROM DIRECTORY")
    teamid = readint("Enter Team ID to remove: ", 1, 9999)
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute("SELECT teamname FROM teamstats WHERE teamid = %s", (teamid,))
    row = cursor.fetchone()
    if not row:
        print("No team found with ID:", teamid)
        cursor.close()
        conn.close()
        return
    confirm = readtext("Type YES to confirm removing " + row[0] + ": ")
    if confirm.upper() == "YES":
        cursor.execute("DELETE FROM teamstats WHERE teamid = %s", (teamid,))
        conn.commit()
        print("Removed team:", row[0])
    else:
        print("Action cancelled.")
    cursor.close()
    conn.close()

def teammenu():
    while True:
        printheader("HOOPSDATA / TEAMS AND RATINGS")
        print("1. Browse All Finals Teams")
        print("2. Browse Teams by Season Years")
```

```
        print("3. Browse Teams by Conference")
        print("4. Search Teams by Franchise or Coach")
        print("5. Top 15 Highest Scoring Teams")
        print("6. Top 15 Best Defensive Teams")
        print("7. Compare Two Teams Side by Side")
        print("8. Add a New Team")
        print("9. Edit Team Ratings")
        print("10. Change Team Head Coach")
        print("11. Remove a Team")
        print("0. Back to Home Page")
        choice = readint("Choose an option (0 to 11): ", 0, 11)
        if choice == 1:
            viewallteams()
            pausescreen()
        elif choice == 2:
            viewteamsbyyear()
            pausescreen()
        elif choice == 3:
            viewteamsbyconference()
            pausescreen()
        elif choice == 4:
            searchteambyname()
            pausescreen()
        elif choice == 5:
            topoffensiveteams()
            pausescreen()
        elif choice == 6:
            topdefensiveteams()
            pausescreen()
        elif choice == 7:
            comparetwoteams()
            pausescreen()
        elif choice == 8:
            addnewteam()
            pausescreen()
        elif choice == 9:
            updateteamratings()
            pausescreen()
        elif choice == 10:
            updateteamcoach()
            pausescreen()
        elif choice == 11:
            deleteteamrecord()
            pausescreen()
        elif choice == 0:
            break

def viewallseries():
    printheader("NBA FINALS SERIES ARCHIVE (1991 TO 2026)")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT year, champion, runnerup, seriesresult, finalsmvp, totalgames, avgattendance "
        "FROM serieshistory ORDER BY year DESC"
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
```

```
    headers = ["Year", "Champion", "Runner Up", "Result", "Finals MVP", "Games", "Crowd"]
    widths = [4, 21, 22, 6, 23, 5, 6]
    printtable(headers, widths, rows)

def viewseriesbyyearrange():
    printheader("BROWSE FINALS SERIES BY YEARS")
    startyear = readint("Enter Start Year: ", 1950, 2035)
    endyear = readint("Enter End Year: ", startyear, 2035)
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT year, champion, runnerup, seriesresult, finalsmvp, totalgames, avgattendance "
        "FROM serieshistory WHERE year BETWEEN %s AND %s ORDER BY year DESC",
        (startyear, endyear)
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["Year", "Champion", "Runner Up", "Result", "Finals MVP", "Games", "Crowd"]
    widths = [4, 21, 22, 6, 23, 5, 6]
    printtable(headers, widths, rows)

def searchseriesbyteam():
    printheader("SEARCH FINALS SERIES BY FRANCHISE")
    team = readtext("Enter Franchise Name (for example, Knicks, Spurs, Lakers): ")
    pattern = "%" + team + "%"
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT year, champion, runnerup, seriesresult, finalsmvp, totalgames "
        "FROM serieshistory WHERE champion LIKE %s OR runnerup LIKE %s ORDER BY year DESC",
        (pattern, pattern)
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["Year", "Champion", "Runner Up", "Result", "Finals MVP", "Games"]
    widths = [4, 21, 22, 6, 23, 5]
    printtable(headers, widths, rows)

def viewgame7series():
    printheader("CLASSIC 7 GAME FINALS SHOWDOWNS")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT year, champion, runnerup, seriesresult, finalsmvp, avgattendance "
        "FROM serieshistory WHERE totalgames = 7 ORDER BY year DESC"
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["Year", "Champion", "Runner Up", "Result", "Finals MVP", "Crowd"]
    widths = [4, 21, 22, 6, 23, 6]
    printtable(headers, widths, rows)

def viewsweepseries():
    printheader("NBA FINALS 4 GAME SWEEPS")
```

```
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT year, champion, runnerup, seriesresult, finalsmvp, avgattendance "
        "FROM serieshistory WHERE totalgames = 4 ORDER BY year DESC"
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["Year", "Champion", "Runner Up", "Result", "Finals MVP", "Crowd"]
    widths = [4, 21, 22, 6, 23, 6]
    printtable(headers, widths, rows)

def championshipleaderboard():
    printheader("CHAMPIONSHIPS WON BY FRANCHISE")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT champion, COUNT(*) AS titles, MIN(year) AS firsttitle, MAX(year) AS lasttitle "
        "FROM serieshistory GROUP BY champion ORDER BY titles DESC, lasttitle DESC"
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["Franchise Name", "Titles Won", "First Title", "Latest Title"]
    widths = [24, 12, 12, 12]
    printtable(headers, widths, rows)

def finalsappearancesleaderboard():
    printheader("TOTAL FINALS APPEARANCES BY FRANCHISE")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute("SELECT champion, runnerup FROM serieshistory")
    allrows = cursor.fetchall()
    cursor.close()
    conn.close()

    counts = {}
    wins = {}
    losses = {}
    for ch, ru in allrows:
        counts[ch] = counts.get(ch, 0) + 1
        wins[ch] = wins.get(ch, 0) + 1
        losses[ch] = losses.get(ch, 0)
        counts[ru] = counts.get(ru, 0) + 1
        wins[ru] = wins.get(ru, 0)
        losses[ru] = losses.get(ru, 0) + 1

    tabledata = []
    for team in counts:
        total = counts[team]
        w = wins[team]
        l = losses[team]
        winpct = round((w / total) * 100, 1)
        tabledata.append((team, total, w, l, str(winpct) + "%"))

    tabledata.sort(key=lambda item: (item[1], item[2]), reverse=True)
```

```
    headers = ["Franchise Name", "Appearances", "Finals Wins", "Finals Losses", "Win Rate"]
    widths = [24, 12, 12, 13, 9]
    printtable(headers, widths, tabledata)

def addseriesrecord():
    printheader("ADD A FINALS SERIES")
    year = readint("Enter Finals Year (1950 to 2035): ", 1950, 2035)
    champion = readtext("Enter Champion Franchise Name: ")
    runnerup = readtext("Enter Runner Up Franchise Name: ")
    loserwins = readint("Enter games won by Runner Up (0 to 3): ", 0, 3)
    seriesresult = "4 to " + str(loserwins)
    totalgames = 4 + loserwins
    finalsmvp = readtext("Enter Finals MVP Winner: ")
    avgattendance = readint("Enter Average Arena Attendance (0 to 30000): ", 0, 30000)

    conn = getdbconnection()
    cursor = conn.cursor()
    try:
        cursor.execute(
            "INSERT INTO serieshistory VALUES (%s, %s, %s, %s, %s, %s, %s)",
            (year, champion, runnerup, seriesresult, finalsmvp, totalgames, avgattendance)
        )
        conn.commit()
        print("Added Finals series for year", year)
    except Exception as e:
        print("Could not add Finals series:", e)
    cursor.close()
    conn.close()

def updateseriesrecord():
    printheader("EDIT A FINALS SERIES")
    year = readint("Enter Finals Year to edit: ", 1950, 2035)
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute("SELECT champion, runnerup, seriesresult, finalsmvp FROM serieshistory WHERE
year = %s", (year,))
    row = cursor.fetchone()
    if not row:
        print("No Finals series found for year:", year)
        cursor.close()
        conn.close()
        return
    print("Selected Year:", year, "| Champion:", row[0], "| Runner Up:", row[1], "| MVP:", row[3])
    champion = readtext("Enter Updated Champion Name: ")
    runnerup = readtext("Enter Updated Runner Up Name: ")
    loserwins = readint("Enter games won by Runner Up (0 to 3): ", 0, 3)
    seriesresult = "4 to " + str(loserwins)
    totalgames = 4 + loserwins
    finalsmvp = readtext("Enter Updated Finals MVP Name: ")
    avgattendance = readint("Enter Updated Average Attendance: ", 0, 30000)

    cursor.execute(
        "UPDATE serieshistory SET champion = %s, runnerup = %s, seriesresult = %s, "
        "finalsmvp = %s, totalgames = %s, avgattendance = %s WHERE year = %s",
        (champion, runnerup, seriesresult, finalsmvp, totalgames, avgattendance, year)
    )
    conn.commit()
```

```
    print("Updated Finals series for year", year)
    cursor.close()
    conn.close()

def deleteseriesrecord():
    printheader("REMOVE A FINALS SERIES")
    year = readint("Enter Finals Year to remove: ", 1950, 2035)
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute("SELECT champion, runnerup FROM serieshistory WHERE year = %s", (year,))
    row = cursor.fetchone()
    if not row:
        print("No Finals series found for year:", year)
        cursor.close()
        conn.close()
        return
    confirm = readtext("Type YES to remove the " + str(year) + " Finals (" + row[0] + " vs " +
row[1] + "): ")
    if confirm.upper() == "YES":
        cursor.execute("DELETE FROM serieshistory WHERE year = %s", (year,))
        conn.commit()
        print("Removed Finals series for year", year)
    else:
        print("Action cancelled.")
    cursor.close()
    conn.close()

def seriesmenu():
    while True:
        printheader("HOOPSDATA / FINALS SERIES ARCHIVE")
        print("1. Browse All Finals Series")
        print("2. Browse Series by Year Range")
        print("3. Search Series by Franchise")
        print("4. Classic Game 7 Showdowns")
        print("5. 4 Game Finals Sweeps")
        print("6. Championships by Franchise")
        print("7. Total Finals Appearances by Franchise")
        print("8. Add a Finals Series")
        print("9. Edit a Finals Series")
        print("10. Remove a Finals Series")
        print("0. Back to Home Page")
        choice = readint("Choose an option (0 to 10): ", 0, 10)
        if choice == 1:
            viewallseries()
            pausescreen()
        elif choice == 2:
            viewseriesbyyearrange()
            pausescreen()
        elif choice == 3:
            searchseriesbyteam()
            pausescreen()
        elif choice == 4:
            viewgame7series()
            pausescreen()
        elif choice == 5:
            viewsweepseries()
            pausescreen()
        elif choice == 6:
```

```
            championshipleaderboard()
            pausescreen()
        elif choice == 7:
            finalsappearancesleaderboard()
            pausescreen()
        elif choice == 8:
            addseriesrecord()
            pausescreen()
        elif choice == 9:
            updateseriesrecord()
            pausescreen()
        elif choice == 10:
            deleteseriesrecord()
            pausescreen()
        elif choice == 0:
            break

def viewallmvps():
    printheader("NBA FINALS MVP HUB (1991 TO 2026)")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT year, playername, team, ppg, rpg, apg, fgpct "
        "FROM mvpstats ORDER BY year DESC"
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["Year", "Player Name", "Team", "PPG", "RPG", "APG", "FG Pct"]
    widths = [4, 23, 21, 5, 5, 5, 6]
    printtable(headers, widths, rows)

def searchmvpbyplayer():
    printheader("SEARCH FINALS MVPS BY PLAYER")
    name = readtext("Enter player name: ")
    pattern = "%" + name + "%"
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT year, playername, team, ppg, rpg, apg, fgpct "
        "FROM mvpstats WHERE playername LIKE %s ORDER BY year ASC",
        (pattern,)
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["Year", "Player Name", "Team", "PPG", "RPG", "APG", "FG Pct"]
    widths = [4, 23, 21, 5, 5, 5, 6]
    printtable(headers, widths, rows)

def filtermvpsbyppg():
    printheader("BROWSE FINALS MVPS BY SCORING AVERAGE")
    minppg = readfloat("Enter Minimum Points Per Game (10.0 to 45.0): ", 10.0, 45.0)
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT year, playername, team, ppg, rpg, apg, fgpct "
```

```
        "FROM mvpstats WHERE ppg >= %s ORDER BY ppg DESC",
        (minppg,)
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["Year", "Player Name", "Team", "PPG", "RPG", "APG", "FG Pct"]
    widths = [4, 23, 21, 5, 5, 5, 6]
    printtable(headers, widths, rows)

def multiplemvpwinners():
    printheader("MULTI TIME FINALS MVP WINNERS")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT playername, COUNT(*) AS awards, ROUND(AVG(ppg), 1) AS avgppg, "
        "ROUND(AVG(rpg), 1) AS avgrpg, ROUND(AVG(apg), 1) AS avgapg "
        "FROM mvpstats GROUP BY playername HAVING COUNT(*) > 1 ORDER BY awards DESC, avgppg DESC"
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["Player Name", "MVP Awards", "Avg PPG", "Avg RPG", "Avg APG"]
    widths = [24, 12, 9, 9, 9]
    printtable(headers, widths, rows)

def comparetwomvps():
    printheader("HEAD TO HEAD FINALS MVP COMPARISON")
    year1 = readint("Enter First Finals Year (1991 to 2035): ", 1950, 2035)
    year2 = readint("Enter Second Finals Year (1991 to 2035): ", 1950, 2035)
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute("SELECT year, playername, team, ppg, rpg, apg, fgpct FROM mvpstats WHERE year =
%s", (year1,))
    mvp1 = cursor.fetchone()
    cursor.execute("SELECT year, playername, team, ppg, rpg, apg, fgpct FROM mvpstats WHERE year =
%s", (year2,))
    mvp2 = cursor.fetchone()
    cursor.close()
    conn.close()

    if not mvp1 or not mvp2:
        print("One or both Finals years could not be found.")
        return

    headers = ["Stat Category", str(mvp1[0]) + " " + mvp1[1], str(mvp2[0]) + " " + mvp2[1]]
    widths = [20, 28, 28]
    score1 = round(float(mvp1[3]) + (float(mvp1[4]) * 1.2) + (float(mvp1[5]) * 1.5), 2)
    score2 = round(float(mvp2[3]) + (float(mvp2[4]) * 1.2) + (float(mvp2[5]) * 1.5), 2)
    rows = [
        ("Team", mvp1[2], mvp2[2]),
        ("Points Per Game", mvp1[3], mvp2[3]),
        ("Rebounds Per Game", mvp1[4], mvp2[4]),
        ("Assists Per Game", mvp1[5], mvp2[5]),
        ("Field Goal Pct", str(mvp1[6]) + "%", str(mvp2[6]) + "%"),
        ("Overall Player Score", score1, score2)
    ]
```

```
    printtable(headers, widths, rows)
    if score1 > score2:
        print("Higher Overall Performance:", mvp1[0], mvp1[1])
    elif score2 > score1:
        print("Higher Overall Performance:", mvp2[0], mvp2[1])
    else:
        print("Both MVP performances tied in Overall Player Score!")

def addmvprecord():
    printheader("ADD AN MVP PROFILE")
    year = readint("Enter Finals Year (1950 to 2035): ", 1950, 2035)
    playername = readtext("Enter MVP Player Name: ")
    team = readtext("Enter Team Name: ")
    ppg = readfloat("Enter Points Per Game (0.0 to 60.0): ", 0.0, 60.0)
    rpg = readfloat("Enter Rebounds Per Game (0.0 to 35.0): ", 0.0, 35.0)
    apg = readfloat("Enter Assists Per Game (0.0 to 25.0): ", 0.0, 25.0)
    fgpct = readfloat("Enter Field Goal Percentage (10.0 to 90.0): ", 10.0, 90.0)

    conn = getdbconnection()
    cursor = conn.cursor()
    try:
        cursor.execute(
            "INSERT INTO mvpstats VALUES (%s, %s, %s, %s, %s, %s, %s)",
            (year, playername, team, ppg, rpg, apg, fgpct)
        )
        conn.commit()
        print("Added Finals MVP profile for", year)
    except Exception as e:
        print("Could not add MVP profile:", e)
    cursor.close()
    conn.close()

def updatemvprecord():
    printheader("EDIT MVP STATS")
    year = readint("Enter Finals Year to edit: ", 1950, 2035)
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute("SELECT playername, ppg, rpg, apg, fgpct FROM mvpstats WHERE year = %s",
(year,))
    row = cursor.fetchone()
    if not row:
        print("No MVP profile found for year:", year)
        cursor.close()
        conn.close()
        return
    print("Selected Year:", year, "| Player:", row[0], "| PPG:", row[1], "RPG:", row[2], "APG:",
row[3])
    playername = readtext("Enter Updated Player Name: ")
    ppg = readfloat("Enter Updated PPG: ", 0.0, 60.0)
    rpg = readfloat("Enter Updated RPG: ", 0.0, 35.0)
    apg = readfloat("Enter Updated APG: ", 0.0, 25.0)
    fgpct = readfloat("Enter Updated FG Percentage: ", 10.0, 90.0)
    cursor.execute(
        "UPDATE mvpstats SET playername = %s, ppg = %s, rpg = %s, apg = %s, fgpct = %s WHERE year
= %s",
        (playername, ppg, rpg, apg, fgpct, year)
    )
```

```
    conn.commit()
    print("Updated MVP profile for year", year)
    cursor.close()
    conn.close()

def deletemvprecord():
    printheader("REMOVE AN MVP PROFILE")
    year = readint("Enter Finals Year to remove: ", 1950, 2035)
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute("SELECT playername FROM mvpstats WHERE year = %s", (year,))
    row = cursor.fetchone()
    if not row:
        print("No MVP profile found for year:", year)
        cursor.close()
        conn.close()
        return
    confirm = readtext("Type YES to remove the MVP profile for " + str(year) + " (" + row[0] + "):
")
    if confirm.upper() == "YES":
        cursor.execute("DELETE FROM mvpstats WHERE year = %s", (year,))
        conn.commit()
        print("Removed MVP profile for year", year)
    else:
        print("Action cancelled.")
    cursor.close()
    conn.close()

def mvpmenu():
    while True:
        printheader("HOOPSDATA / FINALS MVP HUB")
        print("1. Browse All Finals MVPs")
        print("2. Search MVPs by Player Name")
        print("3. Browse MVPs by Scoring Average")
        print("4. Multi Time Finals MVP Winners")
        print("5. Compare Two Finals MVP Performances")
        print("6. Add an MVP Profile")
        print("7. Edit MVP Stats")
        print("8. Remove an MVP Profile")
        print("0. Back to Home Page")
        choice = readint("Choose an option (0 to 8): ", 0, 8)
        if choice == 1:
            viewallmvps()
            pausescreen()
        elif choice == 2:
            searchmvpbyplayer()
            pausescreen()
        elif choice == 3:
            filtermvpsbyppg()
            pausescreen()
        elif choice == 4:
            multiplemvpwinners()
            pausescreen()
        elif choice == 5:
            comparetwomvps()
            pausescreen()
        elif choice == 6:
            addmvprecord()
```

```
            pausescreen()
        elif choice == 7:
            updatemvprecord()
            pausescreen()
        elif choice == 8:
            deletemvprecord()
            pausescreen()
        elif choice == 0:
            break

def viewallarenas():
    printheader("NBA ARENAS AND VENUES DIRECTORY")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT arenaid, arenaname, city, franchise, capacity, openedyear "
        "FROM arenainfo ORDER BY capacity DESC"
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["ID", "Arena Name", "City", "Home Team", "Capacity", "Opened"]
    widths = [2, 24, 14, 22, 8, 6]
    printtable(headers, widths, rows)

def searcharenabycity():
    printheader("SEARCH ARENAS BY CITY OR TEAM")
    keyword = readtext("Enter city, team, or arena name: ")
    pattern = "%" + keyword + "%"
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT arenaid, arenaname, city, franchise, capacity, openedyear "
        "FROM arenainfo WHERE city LIKE %s OR franchise LIKE %s OR arenaname LIKE %s",
        (pattern, pattern, pattern)
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["ID", "Arena Name", "City", "Home Team", "Capacity", "Opened"]
    widths = [2, 24, 14, 22, 8, 6]
    printtable(headers, widths, rows)

def filterarenasbycapacity():
    printheader("BROWSE ARENAS BY SEATING CAPACITY")
    mincap = readint("Enter Minimum Seating Capacity (15000 to 25000): ", 15000, 25000)
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT arenaid, arenaname, city, franchise, capacity, openedyear "
        "FROM arenainfo WHERE capacity >= %s ORDER BY capacity DESC",
        (mincap,)
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["ID", "Arena Name", "City", "Home Team", "Capacity", "Opened"]
```

```
    widths = [2, 24, 14, 22, 8, 6]
    printtable(headers, widths, rows)

def oldestarenasreport():
    printheader("CLASSIC AND HISTORIC NBA ARENAS")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT arenaid, arenaname, city, franchise, openedyear, (2026 - openedyear) AS age,
capacity "
        "FROM arenainfo ORDER BY openedyear ASC"
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["ID", "Arena Name", "City", "Home Team", "Opened", "Years", "Capacity"]
    widths = [2, 24, 14, 22, 6, 6, 8]
    printtable(headers, widths, rows)

def addarenarecord():
    printheader("REGISTER A NEW NBA ARENA")
    arenaname = readtext("Enter Arena Name: ")
    city = readtext("Enter City Name: ")
    franchise = readtext("Enter Home Team Name: ")
    capacity = readint("Enter Seating Capacity (10000 to 35000): ", 10000, 35000)
    openedyear = readint("Enter Year Opened (1950 to 2035): ", 1950, 2035)

    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute("SELECT COALESCE(MAX(arenaid), 0) + 1 FROM arenainfo")
    newid = cursor.fetchone()[0]
    try:
        cursor.execute(
            "INSERT INTO arenainfo VALUES (%s, %s, %s, %s, %s, %s)",
            (newid, arenaname, city, franchise, capacity, openedyear)
        )
        conn.commit()
        print("Registered arena:", arenaname, "with ID:", newid)
    except Exception as e:
        print("Could not register arena:", e)
    cursor.close()
    conn.close()

def updatearenacapacity():
    printheader("EDIT ARENA DETAILS")
    arenaid = readint("Enter Arena ID to edit: ", 1, 9999)
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute("SELECT arenaname, capacity FROM arenainfo WHERE arenaid = %s", (arenaid,))
    row = cursor.fetchone()
    if not row:
        print("No arena found with ID:", arenaid)
        cursor.close()
        conn.close()
        return
    print("Selected Arena:", row[0], "| Current Capacity:", row[1])
    newname = readtext("Enter Updated Arena Name: ")
```

```
    newcap = readint("Enter Updated Seating Capacity (10000 to 35000): ", 10000, 35000)
    cursor.execute(
        "UPDATE arenainfo SET arenaname = %s, capacity = %s WHERE arenaid = %s",
        (newname, newcap, arenaid)
    )
    conn.commit()
    print("Updated details for arena ID", arenaid)
    cursor.close()
    conn.close()

def deletearenarecord():
    printheader("REMOVE AN ARENA")
    arenaid = readint("Enter Arena ID to remove: ", 1, 9999)
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute("SELECT arenaname FROM arenainfo WHERE arenaid = %s", (arenaid,))
    row = cursor.fetchone()
    if not row:
        print("No arena found with ID:", arenaid)
        cursor.close()
        conn.close()
        return
    confirm = readtext("Type YES to remove arena " + row[0] + ": ")
    if confirm.upper() == "YES":
        cursor.execute("DELETE FROM arenainfo WHERE arenaid = %s", (arenaid,))
        conn.commit()
        print("Removed arena:", row[0])
    else:
        print("Action cancelled.")
    cursor.close()
    conn.close()

def arenamenu():
    while True:
        printheader("HOOPSDATA / NBA ARENAS AND VENUES")
        print("1. Browse All Arenas by Capacity")
        print("2. Search Arenas by City or Team")
        print("3. Browse Arenas by Minimum Capacity")
        print("4. Classic and Historic NBA Arenas")
        print("5. Register a New Arena")
        print("6. Edit Arena Details")
        print("7. Remove an Arena")
        print("0. Back to Home Page")
        choice = readint("Choose an option (0 to 7): ", 0, 7)
        if choice == 1:
            viewallarenas()
            pausescreen()
        elif choice == 2:
            searcharenabycity()
            pausescreen()
        elif choice == 3:
            filterarenasbycapacity()
            pausescreen()
        elif choice == 4:
            oldestarenasreport()
            pausescreen()
        elif choice == 5:
            addarenarecord()
```

```
            pausescreen()
        elif choice == 6:
            updatearenacapacity()
            pausescreen()
        elif choice == 7:
            deletearenarecord()
            pausescreen()
        elif choice == 0:
            break

def getteambyid(teamid):
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT teamid, teamname, offrating, defrating, pace, coach "
        "FROM teamstats WHERE teamid = %s",
        (teamid,)
    )
    row = cursor.fetchone()
    cursor.close()
    conn.close()
    return row

def savetosimlogs(teama, teamb, winner, games, margin):
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute("SELECT COALESCE(MAX(simid), 0) + 1 FROM simlogs")
    newid = cursor.fetchone()[0]
    now = datetime.now().strftime("%Y/%m/%d %H:%M")
    cursor.execute(
        "INSERT INTO simlogs VALUES (%s, %s, %s, %s, %s, %s, %s)",
        (newid, now, teama, teamb, winner, int(games), float(margin))
    )
    conn.commit()
    cursor.close()
    conn.close()

def runratingsimulation():
    printheader("HEAD TO HEAD SERIES MATCHUP PREDICTOR")
    viewallteams()
    print()
    ida = readint("Enter Team ID for Team A: ", 1, 9999)
    idb = readint("Enter Team ID for Team B: ", 1, 9999)
    teama = getteambyid(ida)
    teamb = getteambyid(idb)

    if not teama or not teamb:
        print("One or both Team IDs could not be found.")
        return

    namea = teama[1]
    offa = float(teama[2])
    defa = float(teama[3])
    pacea = float(teama[4])
    coacha = teama[5]

    nameb = teamb[1]
```

```
    offb = float(teamb[2])
    defb = float(teamb[3])
    paceb = float(teamb[4])
    coachb = teamb[5]

    expectedpace = round((pacea + paceb) / 2.0, 2)
    scorea = round(((offa + defb) / 2.0) * (expectedpace / 100.0), 1)
    scoreb = round(((offb + defa) / 2.0) * (expectedpace / 100.0), 1)
    neta = round(offa - defa, 2)
    netb = round(offb - defb, 2)
    diff = round(abs(scorea - scoreb), 2)

    if scorea > scoreb:
        winner = namea
    elif scoreb > scorea:
        winner = nameb
    else:
        winner = namea if neta >= netb else nameb

    if diff >= 6.0:
        predictedgames = 4
    elif diff >= 3.5:
        predictedgames = 5
    elif diff >= 1.5:
        predictedgames = 6
    else:
        predictedgames = 7

    headers = ["Comparison Metric", namea, nameb]
    widths = [24, 27, 27]
    rows = [
        ("Head Coach", coacha, coachb),
        ("Offensive Rating", offa, offb),
        ("Defensive Rating", defa, defb),
        ("Overall Net Rating", neta, netb),
        ("Season Pace", pacea, paceb),
        ("Projected Game Score", scorea, scoreb)
    ]
    printtable(headers, widths, rows)
    print("Projected Series Winner:", winner)
    print("Projected Series Length:", predictedgames, "Games")
    print("Projected Scoring Margin:", diff, "Points Per Game")
    savetosimlogs(namea, nameb, winner, predictedgames, diff)
    print("Matchup result saved to your Matchup History.")

def runbestof7playbyplay():
    printheader("7 GAME SERIES PLAY BY PLAY SIMULATOR")
    ida = readint("Enter Team ID for Team A (See Teams page for IDs): ", 1, 9999)
    idb = readint("Enter Team ID for Team B: ", 1, 9999)
    teama = getteambyid(ida)
    teamb = getteambyid(idb)

    if not teama or not teamb:
        print("One or both Team IDs could not be found.")
        return

    namea = teama[1]
```

```
    offa = float(teama[2])
    defa = float(teama[3])
    pacea = float(teama[4])

    nameb = teamb[1]
    offb = float(teamb[2])
    defb = float(teamb[3])
    paceb = float(teamb[4])

    avgpace = (pacea + paceb) / 2.0
    basea = ((offa + defb) / 2.0) * (avgpace / 100.0)
    baseb = ((offb + defa) / 2.0) * (avgpace / 100.0)

    winsa = 0
    winsb = 0
    gamenum = 1
    totalmargin = 0.0
    gamerows = []

    while winsa < 4 and winsb < 4:
        swinga = random.uniform(0.0, 16.0) - 8.0
        swingb = random.uniform(0.0, 16.0) - 8.0
        pts1 = int(round(basea + swinga))
        pts2 = int(round(baseb + swingb))
        if pts1 == pts2:
            pts1 = pts1 + random.choice([2, 3])
        if pts1 > pts2:
            winsa = winsa + 1
            gamewinner = namea
        else:
            winsb = winsb + 1
            gamewinner = nameb
        totalmargin = totalmargin + abs(pts1 - pts2)
        standing = str(winsa) + " to " + str(winsb)
        gamerows.append(("Game " + str(gamenum), pts1, pts2, gamewinner, standing))
        gamenum = gamenum + 1

    headers = ["Game", "Team A Pts", "Team B Pts", "Game Winner", "Series Lead"]
    widths = [6, 12, 12, 27, 11]
    print("Team A:", namea, "| Team B:", nameb)
    printtable(headers, widths, gamerows)

    totalgames = winsa + winsb
    avgmargin = round(totalmargin / totalgames, 2)
    champion = namea if winsa == 4 else nameb
    print("Series Winner:", champion, "in", totalgames, "games!")
    savetosimlogs(namea, nameb, champion, totalgames, avgmargin)
    print("Series result saved to your Matchup History.")

def viewsimlogs():
    printheader("RECENT MATCHUP SIMULATION HISTORY")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT simid, LEFT(simdate, 16), teama, teamb, winner, predictedgames, margin "
        "FROM simlogs ORDER BY simid DESC LIMIT 50"
    )
```

```
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["ID", "Date", "Team A", "Team B", "Winner", "Games", "Margin"]
    widths = [2, 10, 20, 20, 20, 5, 6]
    printtable(headers, widths, rows)

def searchsimlogsbyteam():
    printheader("SEARCH MATCHUP HISTORY BY TEAM")
    keyword = readtext("Enter Team Name: ")
    pattern = "%" + keyword + "%"
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT simid, LEFT(simdate, 16), teama, teamb, winner, predictedgames, margin "
        "FROM simlogs WHERE teama LIKE %s OR teamb LIKE %s ORDER BY simid DESC",
        (pattern, pattern)
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["ID", "Date", "Team A", "Team B", "Winner", "Games", "Margin"]
    widths = [2, 10, 20, 20, 20, 5, 6]
    printtable(headers, widths, rows)

def deletesimlogbyid():
    printheader("REMOVE A MATCHUP FROM HISTORY")
    simid = readint("Enter Matchup History ID to remove: ", 1, 99999)
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute("SELECT teama, teamb FROM simlogs WHERE simid = %s", (simid,))
    row = cursor.fetchone()
    if not row:
        print("No matchup found with ID:", simid)
        cursor.close()
        conn.close()
        return
    cursor.execute("DELETE FROM simlogs WHERE simid = %s", (simid,))
    conn.commit()
    print("Removed matchup ID", simid, ":", row[0], "vs", row[1])
    cursor.close()
    conn.close()

def clearallsimlogs():
    printheader("CLEAR ALL MATCHUP HISTORY")
    confirm = readtext("Type YES to clear your entire Matchup History: ")
    if confirm.upper() == "YES":
        conn = getdbconnection()
        cursor = conn.cursor()
        cursor.execute("TRUNCATE TABLE simlogs")
        conn.commit()
        cursor.close()
        conn.close()
        print("Matchup History has been cleared.")
    else:
        print("Action cancelled.")
```

```
def simulatormenu():
    while True:
        printheader("HOOPSDATA / MATCHUP PREDICTOR AND SIMULATOR")
        print("1. Instant Matchup Predictor")
        print("2. 7 Game Series Play by Play Simulator")
        print("3. View Matchup History")
        print("4. Search Matchup History by Team")
        print("5. Remove a Matchup From History")
        print("6. Clear All Matchup History")
        print("0. Back to Home Page")
        choice = readint("Choose an option (0 to 6): ", 0, 6)
        if choice == 1:
            runratingsimulation()
            pausescreen()
        elif choice == 2:
            runbestof7playbyplay()
            pausescreen()
        elif choice == 3:
            viewsimlogs()
            pausescreen()
        elif choice == 4:
            searchsimlogsbyteam()
            pausescreen()
        elif choice == 5:
            deletesimlogbyid()
            pausescreen()
        elif choice == 6:
            clearallsimlogs()
            pausescreen()
        elif choice == 0:
            break

def championshipseasonbreakdown():
    printheader("CHAMPIONSHIP SEASON BREAKDOWN: TEAMS, COACHES AND MVPS")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT s.year, s.champion, t.offrating, t.defrating, "
        "ROUND(t.offrating - t.defrating, 2) AS netrtg, m.playername, m.ppg, t.coach "
        "FROM serieshistory s "
        "INNER JOIN mvpstats m ON s.year = m.year "
        "INNER JOIN teamstats t ON t.teamname = CONCAT(s.year, ' ', s.champion) "
        "ORDER BY s.year DESC"
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["Year", "Champion", "Off", "Def", "Net", "Finals MVP", "PPG", "Head Coach"]
    widths = [4, 20, 5, 5, 5, 22, 5, 15]
    printtable(headers, widths, rows)

def decadesummaryreport():
    printheader("DECADE BY DECADE LEAGUE TRENDS (1990S TO 2020S)")
    conn = getdbconnection()
    cursor = conn.cursor()
    decades = [
        ("1990s (1991 to 1999)", 1991, 1999),
        ("2000s (2000 to 2009)", 2000, 2009),
```

```
        ("2010s (2010 to 2019)", 2010, 2019),
        ("2020s (2020 to 2026)", 2020, 2026)
    ]
    reportrows = []
    for label, starty, endy in decades:
        cursor.execute(
            "SELECT COUNT(*), ROUND(AVG(offrating), 2), ROUND(AVG(defrating), 2), ROUND(AVG(pace),
2) "
            "FROM teamstats WHERE seasonyear BETWEEN %s AND %s",
            (starty, endy)
        )
        trow = cursor.fetchone()
        cursor.execute(
            "SELECT ROUND(AVG(totalgames), 2), ROUND(AVG(avgattendance), 0) "
            "FROM serieshistory WHERE year BETWEEN %s AND %s",
            (starty, endy)
        )
        srow = cursor.fetchone()
        reportrows.append((
            label,
            trow[0],
            trow[1],
            trow[2],
            trow[3],
            srow[0],
            int(srow[1]) if srow[1] else 0
        ))
    cursor.close()
    conn.close()
    headers = ["Decade Era", "Teams", "Avg Off", "Avg Def", "Avg Pace", "Avg Games", "Avg Crowd"]
    widths = [21, 5, 8, 8, 8, 9, 9]
    printtable(headers, widths, reportrows)

def drawratingchart():
    printheader("VISUAL CHART: HIGHEST RATED FINALS TEAMS OF ALL TIME")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT teamname, ROUND(offrating - defrating, 2) AS netrtg "
        "FROM teamstats ORDER BY netrtg DESC LIMIT 12"
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    for teamname, netrtg in rows:
        barsize = int(float(netrtg) * 3)
        bar = "#" * max(1, barsize)
        print(formatcell(teamname, 28) + " | " + bar + " (" + str(netrtg) + ")")

def drawtitlechart():
    printheader("VISUAL CHART: MOST CHAMPIONSHIPS BY FRANCHISE")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT champion, COUNT(*) AS titles "
        "FROM serieshistory GROUP BY champion ORDER BY titles DESC"
    )
    rows = cursor.fetchall()
```

```
    cursor.close()
    conn.close()
    for team, titles in rows:
        bar = "#" * (int(titles) * 5)
        print(formatcell(team, 25) + " | " + bar + " (" + str(titles) + ")")

def drawmvpscoringchart():
    printheader("VISUAL CHART: HIGHEST SCORING FINALS MVPS")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT year, playername, ppg FROM mvpstats ORDER BY ppg DESC LIMIT 12"
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    for year, player, ppg in rows:
        label = str(year) + " " + player
        barsize = int(float(ppg))
        bar = "#" * barsize
        print(formatcell(label, 28) + " | " + bar + " (" + str(ppg) + " PPG)")

def coachleaderboard():
    printheader("MOST SUCCESSFUL FINALS HEAD COACHES")
    conn = getdbconnection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT coach, COUNT(*) AS appearances, ROUND(AVG(offrating), 2) AS avgoff, "
        "ROUND(AVG(defrating), 2) AS avgdef "
        "FROM teamstats GROUP BY coach ORDER BY appearances DESC, avgoff DESC LIMIT 15"
    )
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
    headers = ["Head Coach", "Finals Appearances", "Avg Offense", "Avg Defense"]
    widths = [24, 20, 12, 12]
    printtable(headers, widths, rows)

def portalcontentoverview():
    printheader("HOOPSDATA PORTAL CONTENT OVERVIEW")
    conn = getdbconnection()
    cursor = conn.cursor()
    sections = [
        ("teamstats", "Teams and Ratings Directory", "Historical Finals Teams"),
        ("serieshistory", "Finals Series Archive", "Completed Championship Series"),
        ("mvpstats", "Finals MVP Hub", "MVP Winner Profiles"),
        ("arenainfo", "NBA Arenas and Venues", "Host Stadiums"),
        ("simlogs", "Matchup Predictor History", "Saved User Simulations")
    ]
    summaryrows = []
    for tname, sectiontitle, category in sections:
        cursor.execute("SELECT COUNT(*) FROM " + tname)
        cnt = cursor.fetchone()[0]
        summaryrows.append((sectiontitle, category, cnt))
    cursor.close()
    conn.close()
    headers = ["Portal Section", "Content Category", "Total Entries"]
```

```
    widths = [28, 30, 14]
    printtable(headers, widths, summaryrows)

def resetdefaultdata():
    printheader("RESTORE DEFAULT PORTAL DATA")
    confirm = readtext("Type RESET to restore all sections to default historical records: ")
    if confirm.upper() == "RESET":
        conn = getdbconnection()
        cursor = conn.cursor()
        cursor.execute("DROP TABLE IF EXISTS teamstats")
        cursor.execute("DROP TABLE IF EXISTS serieshistory")
        cursor.execute("DROP TABLE IF EXISTS mvpstats")
        cursor.execute("DROP TABLE IF EXISTS arenainfo")
        cursor.execute("DROP TABLE IF EXISTS simlogs")
        conn.commit()
        cursor.close()
        conn.close()
        initdatabase()
        print("All portal sections have been restored to default records.")
    else:
        print("Action cancelled.")

def analyticsmenu():
    while True:
        printheader("HOOPSDATA / LEAGUE INSIGHTS AND LEADERBOARDS")
        print("1. Championship Season Breakdown (Teams, Coaches and MVPs)")
        print("2. Decade by Decade League Trends (1990s to 2020s)")
        print("3. Most Successful Finals Head Coaches")
        print("4. Visual Chart: Highest Rated Teams of All Time")
        print("5. Visual Chart: Most Championships by Franchise")
        print("6. Visual Chart: Highest Scoring Finals MVPs")
        print("7. Portal Content Overview")
        print("8. Restore Default Portal Data")
        print("0. Back to Home Page")
        choice = readint("Choose an option (0 to 8): ", 0, 8)
        if choice == 1:
            championshipseasonbreakdown()
            pausescreen()
        elif choice == 2:
            decadesummaryreport()
            pausescreen()
        elif choice == 3:
            coachleaderboard()
            pausescreen()
        elif choice == 4:
            drawratingchart()
            pausescreen()
        elif choice == 5:
            drawtitlechart()
            pausescreen()
        elif choice == 6:
            drawmvpscoringchart()
            pausescreen()
        elif choice == 7:
            portalcontentoverview()
            pausescreen()
        elif choice == 8:
            resetdefaultdata()
```

```
            pausescreen()
        elif choice == 0:
            break

def mainmenu():
    initdatabase()
    while True:
        printheader("HOOPSDATA | OFFICIAL NBA FINALS PORTAL")
        print("1. Teams and Ratings")
        print("2. Finals Series Archive")
        print("3. Finals MVP Hub")
        print("4. NBA Arenas and Venues")
        print("5. Matchup Predictor and Simulator")
        print("6. League Insights and Leaderboards")
        print("0. Log Out and Exit")
        printline(98)
        choice = readint("Where would you like to go? (0 to 6): ", 0, 6)
        if choice == 1:
            teammenu()
        elif choice == 2:
            seriesmenu()
        elif choice == 3:
            mvpmenu()
        elif choice == 4:
            arenamenu()
        elif choice == 5:
            simulatormenu()
        elif choice == 6:
            analyticsmenu()
        elif choice == 0:
            print()
            print("Thank you for visiting HoopsData! Goodbye!")
            break

mainmenu()
```

Output

Case 1: Main Menu & Top 15 Offensive Teams Directory

1.1 Portal Main Menu & Navigation to Teams Directory

```
+=============================================================================================+
|                            HOOPSDATA | OFFICIAL NBA FINALS PORTAL                          |
+=============================================================================================+
1. Teams and Ratings
2. Finals Series Archive
3. Finals MVP Hub
4. NBA Arenas and Venues
5. Matchup Predictor and Simulator
6. League Insights and Leaderboards
0. Log Out and Exit
+=============================================================================================+
Where would you like to go? (0 to 6): 1


+=============================================================================================+
|                                  HOOPSDATA / TEAMS AND RATINGS                              |
+=============================================================================================+
1. Browse All Finals Teams
2. Browse Teams by Season Years
3. Browse Teams by Conference
4. Search Teams by Franchise or Coach
5. Top 15 Highest Scoring Teams
6. Top 15 Best Defensive Teams
7. Compare Two Teams Side by Side
8. Add a New Team
9. Edit Team Ratings
10. Change Team Head Coach
11. Remove a Team
0. Back to Home Page
Choose an option (0 to 11): 5
```

1.2 Top 15 Highest Scoring Finals Teams Query Output

```
+=============================================================================================+
|                              TOP 15 HIGHEST SCORING FINALS TEAMS                            |
+=============================================================================================+
+====+==============================+======+======+=========+=========+=========+==================+
| ID | Team Name                    | Year | Conf | Offense | Defense | Overall | Head Coach       |
+====+==============================+======+======+=========+=========+=========+==================+
| 67 | 2024 Boston Celtics          | 2024 | East | 122.20  | 109.20  | 13.00   | Joe Mazzulla     |
| 70 | 2025 Indiana Pacers          | 2025 | East | 119.80  | 115.10  | 4.70    | Rick Carlisle    |
| 71 | 2026 New York Knicks         | 2026 | East | 119.80  | 111.20  | 8.60    | Mike Brown       |
| 69 | 2025 Oklahoma City Thunder   | 2025 | West | 118.50  | 108.20  | 10.30   | Mark Daigneault  |
| 72 | 2026 San Antonio Spurs       | 2026 | West | 118.40  | 110.10  | 8.30    | Mitch Johnson    |
| 65 | 2023 Denver Nuggets          | 2023 | West | 117.60  | 113.50  | 4.10    | Michael Malone   |
| 62 | 2021 Phoenix Suns            | 2021 | West | 117.20  | 111.30  | 5.90    | Monty Williams   |
| 61 | 2021 Milwaukee Bucks         | 2021 | East | 117.20  | 111.00  | 6.20    | Mike Budenholzer |
| 68 | 2024 Dallas Mavericks        | 2024 | West | 117.00  | 114.90  | 2.10    | Jason Kidd       |
| 58 | 2019 Golden State Warriors   | 2019 | West | 115.90  | 109.00  | 6.90    | Steve Kerr       |
| 53 | 2017 Golden State Warriors   | 2017 | West | 115.60  | 102.50  | 13.10   | Steve Kerr       |
| 3  | 1992 Chicago Bulls           | 1992 | East | 115.50  | 104.50  | 11.00   | Phil Jackson     |
| 11 | 1996 Chicago Bulls           | 1996 | East | 115.20  | 101.80  | 13.40   | Phil Jackson     |
| 10 | 1995 Orlando Magic           | 1995 | East | 115.10  | 107.80  | 7.30    | Brian Hill       |
| 1  | 1991 Chicago Bulls           | 1991 | East | 114.60  | 105.20  | 9.40    | Phil Jackson     |
+====+==============================+======+======+=========+=========+=========+==================+
Showing 15 results
```

Case 2: Side-by-Side Team Comparison & Managing Teams (Add, Edit, Delete)

2.1 Head-to-Head Comparison: 1996 Chicago Bulls vs. 2017 Golden State Warriors

```
+=============================================================================================+
|                                  SIDE BY SIDE TEAM COMPARISON                               |
+=============================================================================================+
Enter First Team ID: 11
Enter Second Team ID: 53


+======================+============================+============================+
| Team Attribute       | 1996 Chicago Bulls         | 2017 Golden State Warriors  |
+======================+============================+============================+
| Season Year          | 1996                       | 2017                       |
| Conference           | East                       | West                       |
| Head Coach           | Phil Jackson               | Steve Kerr                 |
| Offensive Rating     | 115.20                     | 115.60                     |
| Defensive Rating     | 101.80                     | 102.50                     |
| Overall Net Rating   | 13.4                       | 13.1                       |
| Game Pace            | 91.10                      | 99.80                      |
+======================+============================+============================+
Showing 7 results
```

2.2 Managing Team Records: Insertion, Updating Ratings, and Record Deletion

```
+=============================================================================================+
|                                    ADD A NEW FINALS TEAM                                    |
+=============================================================================================+
Enter Season Year (1950 to 2035): 2027
Enter Team Franchise Name (for example, New York Knicks): New York Knicks
Select Conference: 1. East  2. West
Enter choice (1 or 2): 1
Enter Offensive Rating (80.0 to 140.0): 121.50
Enter Defensive Rating (80.0 to 140.0): 109.80
Enter Team Pace (75.0 to 120.0): 98.50
Enter Head Coach Name: Mike Brown
Added team: 2027 New York Knicks with ID: 73

+=============================================================================================+
|                                EDIT TEAM RATINGS AND PACE                                   |
+=============================================================================================+
Enter Team ID to edit: 73
Selected Team: 2027 New York Knicks
Current Offense: 121.50 | Current Defense: 109.80 | Current Pace: 98.50
Enter New Offensive Rating (80.0 to 140.0): 122.00
Enter New Defensive Rating (80.0 to 140.0): 109.00
Enter New Pace (75.0 to 120.0): 99.00
Updated ratings for 2027 New York Knicks

+=============================================================================================+
|                              REMOVE A TEAM FROM DIRECTORY                                   |
+=============================================================================================+
Enter Team ID to remove: 73
Type YES to confirm removing 2027 New York Knicks: YES
Removed team: 2027 New York Knicks
```

Case 3: Finals Series Archive & Franchise Leaderboards

3.1 Historical 7-Game Finals Showdowns (1994 to 2025)

CLASSIC 7 GAME FINALS SHOWDOWNS

Year	Champion	Runner Up	Result	Finals MVP	Crowd
2025	Oklahoma City Thunder	Indiana Pacers	4 to 3	Shai Gilgeous Alexander	18200
2016	Cleveland Cavaliers	Golden State Warriors	4 to 3	LeBron James	20200
2013	Miami Heat	San Antonio Spurs	4 to 3	LeBron James	19500
2010	Los Angeles Lakers	Boston Celtics	4 to 3	Kobe Bryant	18900
2005	San Antonio Spurs	Detroit Pistons	4 to 3	Tim Duncan	20400
1994	Houston Rockets	New York Knicks	4 to 3	Hakeem Olajuwon	17800

Showing 6 results

3.2 Franchise Championship Leaderboard (SQL Aggregations)

CHAMPIONSHIPS WON BY FRANCHISE

Franchise Name	Titles Won	First Title	Latest Title
Los Angeles Lakers	6	2000	2020
Chicago Bulls	6	1991	1998
San Antonio Spurs	5	1999	2014
Golden State Warriors	4	2015	2022
Miami Heat	3	2006	2013
Boston Celtics	2	2008	2024
Houston Rockets	2	1994	1995
New York Knicks	1	2026	2026
Oklahoma City Thunder	1	2025	2025
Denver Nuggets	1	2023	2023
Milwaukee Bucks	1	2021	2021
Toronto Raptors	1	2019	2019
Cleveland Cavaliers	1	2016	2016
Dallas Mavericks	1	2011	2011
Detroit Pistons	1	2004	2004

Showing 15 results

3.3 All-Time Finals Appearances & Win Percentages Leaderboard

TOTAL FINALS APPEARANCES BY FRANCHISE (TOP 8)

Franchise Name	Appearances	Finals Wins	Finals Losses	Win Rate
Los Angeles Lakers	9	6	3	66.7%
San Antonio Spurs	7	5	2	71.4%
Miami Heat	7	3	4	42.9%
Chicago Bulls	6	6	0	100.0%
Golden State Warriors	6	4	2	66.7%
Cleveland Cavaliers	5	1	4	20.0%
Boston Celtics	4	2	2	50.0%
New York Knicks	3	1	2	33.3%

Showing 8 results

Case 4: Finals MVP Hub

4.1 Multi-Time Finals MVP Winners (SQL GROUP BY & HAVING Filter)

```
+=============================================================================================+
|                                MULTI TIME FINALS MVP WINNERS                                |
+=============================================================================================+
+==========================+==============+===========+===========+===========+
| Player Name              | MVP Awards   | Avg PPG   | Avg RPG   | Avg APG   |
+==========================+==============+===========+===========+===========+
| Michael Jordan           | 6            | 33.5      | 6.0       | 6.1       |
| LeBron James             | 4            | 28.4      | 11.1      | 8.0       |
| Shaquille O'Neal         | 3            | 35.8      | 14.9      | 3.6       |
| Tim Duncan               | 3            | 24.1      | 15.0      | 3.3       |
| Kevin Durant             | 2            | 32.0      | 9.5       | 6.5       |
| Kobe Bryant              | 2            | 30.5      | 6.8       | 5.7       |
| Hakeem Olajuwon          | 2            | 29.9      | 10.3      | 4.6       |
| Kawhi Leonard            | 2            | 23.2      | 8.1       | 3.1       |
+==========================+==============+===========+===========+===========+
Showing 8 results
```

4.2 Head-to-Head MVP Comparison: 1993 Michael Jordan vs. 2016 LeBron James

```
+=============================================================================================+
|                              HEAD TO HEAD FINALS MVP COMPARISON                             |
+=============================================================================================+
Enter First Finals Year (1991 to 2035): 1993
Enter Second Finals Year (1991 to 2035): 2016

+======================+==============================+==============================+
| Stat Category        | 1993 Michael Jordan          | 2016 LeBron James            |
+======================+==============================+==============================+
| Team                 | Chicago Bulls                | Cleveland Cavaliers          |
| Points Per Game      | 41.0                         | 29.7                         |
| Rebounds Per Game    | 8.5                          | 11.3                         |
| Assists Per Game     | 6.3                          | 8.9                          |
| Field Goal Pct       | 50.8%                        | 49.4%                        |
| Overall Player Score | 60.65                        | 56.61                        |
+======================+==============================+==============================+
Showing 6 results
Higher Overall Performance: 1993 Michael Jordan
```

Case 5: NBA Arenas & Venues Directory

5.1 Historic NBA Arenas Ranked by Age & Modifying Arena Records

```
+=============================================================================================+
|                                CLASSIC AND HISTORIC NBA ARENAS                              |
+=============================================================================================+
+====+=========================+================+=======================+========+========+===========+
| ID | Arena Name              | City           | Home Team             | Opened | Years  | Capacity |
+====+=========================+================+=======================+========+========+===========+
| 16 | Madison Square Garden   | New York       | New York Knicks       | 1968   | 58     | 19812    |
| 20 | Delta Center            | Salt Lake City | Utah Jazz             | 1991   | 35     | 18306    |
| 18 | Footprint Center        | Phoenix        | Phoenix Suns          | 1992   | 34     | 17071    |
| 1  | United Center           | Chicago        | Chicago Bulls         | 1994   | 32     | 20917    |
| 7  | Rocket Mortgage FieldHou | Cleveland      | Cleveland Cavaliers   | 1994   | 32     | 19432    |
| 3  | TD Garden               | Boston         | Boston Celtics        | 1995   | 31     | 19156    |
| 21 | Moda Center             | Portland       | Portland Trail Blazers | 1995   | 31     | 19393    |
| 17 | Wells Fargo Center      | Philadelphia   | Philadelphia 76ers    | 1996   | 30     | 20478    |
| 2  | Crypto.com Arena        | Los Angeles    | Los Angeles Lakers    | 1999   | 27     | 18997    |
| 5  | Kaseya Center           | Miami          | Miami Heat            | 1999   | 27     | 19600    |
+====+=========================+================+=======================+========+========+===========+
Showing 10 results


+=============================================================================================+
|                                      EDIT ARENA DETAILS                                     |
+=============================================================================================+
Enter Arena ID to edit: 16
Selected Arena: Madison Square Garden | Current Capacity: 19812
Enter Updated Arena Name: Madison Square Garden (The World's Most Famous Arena)
Enter Updated Seating Capacity (10000 to 35000): 19812
Updated details for arena ID 16
```

Case 6: Matchup Predictor & 7-Game Play-by-Play Simulator

6.1 Deterministic Matchup Prediction: 2024 Boston Celtics vs. 2026 New York Knicks

```
+=============================================================================================+
|                             HEAD TO HEAD SERIES MATCHUP PREDICTOR                           |
+=============================================================================================+
Enter Team ID for Team A: 67
Enter Team ID for Team B: 71


+=========================+=============================+=============================+
| Comparison Metric       | 2024 Boston Celtics         | 2026 New York Knicks        |
+=========================+=============================+=============================+
| Head Coach              | Joe Mazzulla                | Mike Brown                  |
| Offensive Rating        | 122.20                      | 119.80                      |
| Defensive Rating        | 109.20                      | 111.20                      |
| Overall Net Rating      | 13.0                        | 8.6                         |
| Season Pace             | 97.90                       | 97.80                       |
| Projected Game Score    | 114.2                       | 112.0                       |
+=========================+=============================+=============================+
Showing 6 results
Projected Series Winner: 2024 Boston Celtics
Projected Series Length: 6 Games
Projected Scoring Margin: 2.2 Points Per Game
Matchup result saved to your Matchup History.
```

6.2 Best-of-7 Play-by-Play Series Simulation & Persistent Database Logging

```
+=============================================================================================+
|                              7 GAME SERIES PLAY BY PLAY SIMULATOR                           |
+=============================================================================================+
Enter Team ID for Team A: 11
Enter Team ID for Team B: 52
Team A: 1996 Chicago Bulls | Team B: 2016 Golden State Warriors
+========+==============+==============+============================+=============+
| Game   | Team A Pts   | Team B Pts   | Game Winner                | Series Lead |
+========+==============+==============+============================+=============+
| Game 1 | 108          | 102          | 1996 Chicago Bulls         | 1 to 0      |
| Game 2 | 99           | 105          | 2016 Golden State Warriors | 1 to 1      |
| Game 3 | 104          | 98           | 1996 Chicago Bulls         | 2 to 1      |
| Game 4 | 112          | 107          | 1996 Chicago Bulls         | 3 to 1      |
| Game 5 | 96           | 101          | 2016 Golden State Warriors | 3 to 2      |
| Game 6 | 110          | 103          | 1996 Chicago Bulls         | 4 to 2      |
+========+==============+==============+============================+=============+
Showing 6 results
Series Winner: 1996 Chicago Bulls in 6 games!
Series result saved to your Matchup History.


+=============================================================================================+
|                               RECENT MATCHUP SIMULATION HISTORY                             |
+=============================================================================================+
+====+============+======================+======================+======================+=======+========+
| ID | Date       | Team A               | Team B               | Winner               | Games | Margin |
+====+============+======================+======================+======================+=======+========+
| 2  | 2026/11/07 | 1996 Chicago Bulls   | 2016 Golden State    | 1996 Chicago Bulls   | 6     | 5.83   |
| 1  | 2026/11/07 | 2024 Boston Celtics  | 2026 New York Knicks | 2024 Boston Celtics  | 6     | 2.1    |
+====+============+======================+======================+======================+=======+========+
Showing 2 results


+=============================================================================================+
|                              REMOVE A MATCHUP FROM HISTORY                                  |
+=============================================================================================+
Enter Matchup History ID to remove: 2
Removed matchup ID 2: 1996 Chicago Bulls vs 2016 Golden State Warriors
```

Case 7: League Insights, Multi-Table Joins & ASCII Visual Charts

7.1 Decade-by-Decade League Evolution (1990s to 2020s Analysis)

```
+=============================================================================================+
|                         DECADE BY DECADE LEAGUE TRENDS (1990S TO 2020S)                     |
+=============================================================================================+
+======================+=======+==========+==========+==========+===========+===========+
| Decade Era           | Teams | Avg Off  | Avg Def  | Avg Pace | Avg Games | Avg Crowd |
+======================+=======+==========+==========+==========+===========+===========+
| 1990s (1991 to 1999) | 18    | 110.85   | 103.16   | 92.83    | 5.67      | 19917     |
| 2000s (2000 to 2009) | 20    | 106.96   | 100.97   | 91.17    | 5.40      | 19510     |
| 2010s (2010 to 2019) | 20    | 111.29   | 104.65   | 94.99    | 5.80      | 19715     |
| 2020s (2020 to 2026) | 14    | 116.58   | 110.44   | 98.97    | 5.71      | 16050     |
+======================+=======+==========+==========+==========+===========+===========+
Showing 4 results
```

7.2 Multi-Table INNER JOIN Query: Teams, Head Coaches, and Finals MVPs (2026–2015)

```
+=============================================================================================+
|                      CHAMPIONSHIP SEASON BREAKDOWN: TEAMS, COACHES AND MVPS                 |
+=============================================================================================+
+======+======================+=======+=======+=======+=======================+=======+=================+
| Year | Champion             | Off   | Def   | Net   | Finals MVP            | PPG   | Head Coach      |
+======+======================+=======+=======+=======+=======================+=======+=================+
| 2026 | New York Knicks      | 119.8 | 111.2 | 8.60  | Jalen Brunson         | 32.6  | Mike Brown      |
| 2025 | Oklahoma City Thunde | 118.5 | 108.2 | 10.30 | Shai Gilgeous Alexande | 30.3 | Mark Daigneault |
| 2024 | Boston Celtics       | 122.2 | 109.2 | 13.00 | Jaylen Brown          | 20.8  | Joe Mazzulla    |
| 2023 | Denver Nuggets       | 117.6 | 113.5 | 4.10  | Nikola Jokic          | 30.2  | Michael Malone  |
| 2022 | Golden State Warrior | 112.5 | 106.6 | 5.90  | Stephen Curry         | 31.2  | Steve Kerr      |
| 2021 | Milwaukee Bucks      | 117.2 | 111.0 | 6.20  | Giannis Antetokounmpo | 35.2  | Mike Budenholze |
| 2020 | Los Angeles Lakers   | 112.0 | 105.6 | 6.40  | LeBron James          | 29.8  | Frank Vogel     |
| 2019 | Toronto Raptors      | 113.1 | 107.1 | 6.00  | Kawhi Leonard         | 28.5  | Nick Nurse      |
| 2018 | Golden State Warrior | 113.6 | 107.7 | 5.90  | Kevin Durant          | 28.8  | Steve Kerr      |
| 2017 | Golden State Warrior | 115.6 | 102.5 | 13.10 | Kevin Durant          | 35.2  | Steve Kerr      |
| 2016 | Cleveland Cavaliers  | 110.9 | 104.2 | 6.70  | LeBron James          | 29.7  | Tyronn Lue      |
| 2015 | Golden State Warrior | 111.6 | 101.4 | 10.20 | Andre Iguodala        | 16.3  | Steve Kerr      |
+======+======================+=======+=======+=======+=======================+=======+=================+
Showing 12 results
```

7.3 Visual ASCII Horizontal Bar Charts & Portal Database Content Summary

```
+=============================================================================================+
|                     VISUAL CHART: HIGHEST RATED FINALS TEAMS OF ALL TIME                    |
+=============================================================================================+
1996 Chicago Bulls           | ######################################## (13.40)
2017 Golden State Warriors   | ######################################## (13.10)
2024 Boston Celtics          | ######################################## (13.00)
1997 Chicago Bulls           | ##################################### (12.00)
2008 Boston Celtics          | ################################### (11.30)
1992 Chicago Bulls           | ################################### (11.00)


+=============================================================================================+
|                         VISUAL CHART: MOST CHAMPIONSHIPS BY FRANCHISE                       |
+=============================================================================================+
Chicago Bulls             | ############################## (6)
Los Angeles Lakers        | ############################## (6)
San Antonio Spurs         | ######################### (5)
Golden State Warriors     | ################### (4)
Miami Heat                | ############### (3)


+=============================================================================================+
|                                HOOPSDATA PORTAL CONTENT OVERVIEW                            |
+=============================================================================================+
+==============================+================================+================+
| Portal Section               | Content Category               | Total Entries  |
+==============================+================================+================+
| Teams and Ratings Directory  | Historical Finals Teams        | 72             |
| Finals Series Archive        | Completed Championship Series  | 36             |
| Finals MVP Hub               | MVP Winner Profiles            | 36             |
| NBA Arenas and Venues        | Host Stadiums                  | 22             |
| Matchup Predictor History    | Saved User Simulations         | 1              |
+==============================+================================+================+
Showing 5 results
```

Conclusion

This project successfully demonstrates how Python and MySQL can work together to create a practical, analytical, and interactive sports database management system through the HoopsData Official NBA Finals Portal. Through its structured menu-driven interface and custom ASCII table engine, the application allows users to manage historical Finals teams, explore championship series archives, evaluate Finals MVP statistics, track NBA arena capacities, and simulate head-to-head playoff matchups with persistent logging. The system ensures smooth connectivity between the Python frontend and the `nbafinalsdb` relational database, offering reliable CRUD operations, complex SQL aggregations, multi-table `INNER JOIN` reporting, and automated data validation.

Overall, this project highlights the effectiveness of integrating procedural Python programming with relational database technologies to solve real-world sports data organization and statistical modeling challenges. By combining an intuitive command-line workflow with strong backend SQL logic and algorithmic game simulation, the system provides a complete, accessible, and efficient solution for archiving and analyzing over three decades of NBA Finals history.

Bibliography

1. **Python Official Documentation**
   https://docs.python.org/3/

2. **MySQL 8.0 / 9.0 Reference Manual & Command Line Client Documentation**
   https://dev.mysql.com/doc/

3. **MySQL Connector/Python Developer Guide**
   https://dev.mysql.com/doc/connector-python/en/

4. **GeeksforGeeks (Python–MySQL Database Connectivity)**
   https://www.geeksforgeeks.org/python-mysql/

5. **Basketball-Reference (Historical NBA Finals, Team Ratings & MVP Statistics)**
   https://www.basketball-reference.com/

6. **Computer Science with Python – Class XII (Sumita Arora)**
   Dhanpat Rai & Co.